



ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

Alba Regia Kar
Geoinformatikai Intézet



SZAKDOLGOZAT

THESIS

**Pontfelhők feldolgozása és megjelenítése, saját
fejlesztésű QGIS beépülő modul segítségével**
**Processing and displaying pointclouds alongside vector data
with a custom-built QGIS plugin**

OE-AMK
2026

Hallgató neve: Béres Tamás
Hallgató törzskönyvi száma: ---

Table of contents

1. Introduction.....	1
1.1 The problem: vectors and pointclouds in a browser.....	1
1.2 Analysis of the problem: the old workflow.....	1
1.3 Proposal for the problem.....	2
2. The Solution: PotreeCraft.....	4
2.1 The brief history of the first iteration.....	4
3. QGIS Plugin development: Preparations.....	5
3.1 Setting up the development environment.....	5
3.1.1 Working on Windows.....	5
3.1.2 Working on Linux.....	11
3.1.3 Getting the external dependencies.....	13
4. QGIS Plugin development: Building the plugin.....	16
4.1 The Development loop.....	16
4.1.1 Step 1 – initialize the module with pb_tool create.....	17
4.1.2 Step 2 – Editing the code.....	19
4.1.3 Step 3 – linting and testing.....	19
4.1.4 Step 4 – Compiling the plugin.....	21
4.1.5 Step 5 – Deploying the plugin.....	21
4.1.6 Step 6 – Reloading the plugin.....	22
4.1.7 Step 7 – Testing the changes.....	22
4.1.8 Step 8 – Shipping the plugin with pb_tool zip.....	22
4.1.9 Getting everything ready in one place.....	23
4.2 Architectural overview of the PotreeCraft project.....	25

4.2.1 QGIS Plugin loader.....	26
4.2.2 pb_tool.cfg.....	26
4.2.3 metadata.txt.....	26
4.2.4 __init__.py.....	26
4.2.5 potreecraft.py.....	26
4.2.6 resources.py.....	27
4.2.7 resources.qrc.....	27
4.2.8 plugin icon file.....	28
4.2.9 potreecraft_dialog.py.....	28
4.2.10 potreecraft_dialog_base.ui.....	29
4.2.11 potreecraft_core.py.....	29
4.2.12 potreecraft_lasreader.py.....	29
4.2.13 LAStools: blast2dem.....	30
4.2.14 jslibs folder.....	31
4.2.15 PotreeConverter.....	31
4.2.16 QGIS API.....	31
4.2.17 project output folder.....	31
4.2.18 http server.....	32
4.3 Interaction between the modules.....	32
4.3.1 Raster import sequence.....	32
4.3.2 Vector export sequence.....	33
4.3.3 Compile project sequence.....	35
5. QGIS Plugin development: Testing the tool.....	36
5.1 Testing the tool: the sample “roadsection” dataset.....	36

5.1.1 Pointcloud data.....	36
5.1.2 Vector data.....	38
5.1.3 Putting the project together.....	38
5.2 Testing the tool: Self-collected dataset on Határi dúlő, Pomáz.....	43
5.2.1 Pointcloud data.....	43
5.2.2 Vector data.....	44
6. Conclusion.....	53
6.1 The direction of further development.....	53
6.1.1 What is still missing.....	53
6.1.2 Known bugs.....	54
6.1.3 Direction of development.....	54
7. Summary.....	56
7.1 Summary.....	56
7.2 Összefoglaló (Summary, Hungarian).....	56
8. Bibliography.....	57
9. Table of references.....	58
A. Appendix.....	62
A.I Class method descriptions.....	62
A.I.I potreecraft.py.....	62
A.I.II potreecraft_dialog.py.....	64
A.I.III potreecraft_geojson_reader.py.....	78
A.I.IV potreecraft_lasreader.py.....	80
A.I.V potreecraft_core.py.....	82
A.II JSON and CSV data.....	84

A.II.I Hatri-dl-3-1-2026-cameras.json.....	84
A.II.II Hatri-dl-3-1-2026-camera-params.json.....	84
A.II.III Hatri-dl-3-1-2026-camera-shots.geojson.....	85
A.II.IV overlap_report.csv.....	103

1. INTRODUCTION

Anyone who has had the chance to interact with me on a professional level knows that my career path has not been the most straightforward. This applies both to my years in education and to my time on the job market. My first exposure to computer programming came in high school; however, despite that early experience, I later pursued a BSc in Earth Sciences, specializing in geology. The second turning point came shortly before graduation, when the summer internship I secured was not related to the oil industry, as I had initially expected, but instead involved a GIS project focused on precision agriculture. This few-month-long experience ultimately became a key factor in helping me secure my first job as a fresh graduate. It was there that I encountered a problem—and began developing a solution—which I will discuss in this document.

1.1 The problem: vectors and pointclouds in a browser

During my first year as a junior GIS specialist, my tasks were mostly related to the traditional way of working with point clouds—manual evaluation, importing and exporting hand-drawn vector layers, collecting and cleaning metadata, and assisting my surveyor and cartographer colleagues at various stages of the map-making process.

Sometime during my second year, we encountered the question of how to effectively present the progress of our projects not only to senior management but also to our business partners. After some discussion, we decided to start using Potree[1]. This open source software solution was the thesis project of Markus Schütz in 2016, and it slowly became into an industry-wide used tool with quite a few forks and integrations.

1.2 Analysis of the problem: the old workflow

In order to have a pointcloud we can show in this environment, we need to convert our pointcloud to the octree structure, discussed in chapter 3.3 of Schütz's thesis[1]. This can be done with the following command:

```
PotreeConverter [input files] -o [output directory] --generate-page
[page name] --output-format [output format] --projection [projection
string]
```

Alongside the trivial input-output parameters, we can see a “--generate-page”, which is an important flag here. Without this, PotreeConverter only creates the octree structure data format, but if this is present, we also end up with a boilerplate .html file, which contains a basic potree scene in a viewer, allowing us to see our pointcloud.

My daily tasks were to further edit these pointclouds, and expand upon this boilerplate code section generated by the converter, such as the one in Listing 1.1.

Since, in most companies, land surveyors do not typically work with JavaScript on a daily basis, it is not a skill set they naturally acquire. To the best of my knowledge, the use of such enriched Potree scenes with our vector datasets ceased not long after I left the company.

1.3 Proposal for the problem

The idea for this project was born not long after I switched workplaces. I realized that, in theory, I could help my former colleagues by providing a tool with an easy-to-use graphical interface to automate the JavaScript coding process.

Building such an interface from scratch seemed too ambitious, as it would require solving everything from offering graphical feedback in a 2D/3D preview to handling various file formats, as well as designing a user interface that is not difficult to use.

Since all of my former colleagues are experienced users of various GIS software packages, including QGIS, I thought it would be an interesting idea to offload some of these challenges to an already well-established and thoroughly tested system. QGIS offers excellent support for plugins, and there are many high-quality resources available online for working with Python–QGIS integrations, such as the PyQGIS Developer Cookbook[2]. The goal was to create a small, easy-to-use interface provided by the plugin, so users do not have to worry about using parameter lists in console commands, and also to

provide a way for them to see what they are trying to pull together into one potree scene, using QGIS itself as a springboard for a potree project.

```
<script>
  window.viewer = new Potree.Viewer(document.getEle-
mentById("potree_render_area"));
  viewer.setEDLEnabled(true);
  viewer.setFOV(60);
  viewer.setPointBudget(2_000_000);
  <!-- INCLUDE SETTINGS HERE -->
  viewer.loadSettingsFromURL();
  viewer.setDescription("");
  viewer.loadGUI(() => {
  viewer.setLanguage('en');
    $("#menu_appearance").next().show();
    $("#menu_tools").next().show();
    $("#menu_clipping").next().show();
    viewer.toggleSidebar();
  });
  Potree.loadPointCloud("./pointclouds/test_project/metadata.json",
"test_project", e => {
    let scene = viewer.scene;
    let pointcloud = e.pointcloud;
    let material = pointcloud.material;
    material.size = 1;
    material.pointSizeType = Potree.PointSizeType.ADAPTIVE;
    material.shape = Potree.PointShape.SQUARE;
    material.activeAttributeName = "rgba";
    scene.addPointCloud(pointcloud);
    viewer.fitToScreen();
  });
</script>
```

Listing 1.1: The main script section of the potree boilerplate code.

2. THE SOLUTION: POTREECRAFT

2.1 The brief history of the first iteration

The first lines of code for this project appeared on GitHub on November 12, 2019, and within a week I had a more or less functional proof of concept available for download and testing. However, as this was a Python project that I was using as a portfolio piece to position myself as a junior Python software developer, it is obvious to anyone who dares to explore the commit history that my coding skills were, at best, “suboptimal,” and the entire project was held together by digital duct tape.

As issues within the system accumulated, and unstable features were built on top of equally unstable foundations, I was eventually driven to abandon maintenance of the project around August 2020. The project then remained dormant for approximately five years.

In April 2025, I received an email from Dr. Karol Kwiatek asking whether the plugin was still in development. We began exchanging emails about Potree and my old project, which led me to reconsider the concept behind it once again.

Many days have passed since then, and with nearly five years of software development experience, I now believe I possess the skill set required to turn this plugin into something solid, reliable, and genuinely useful for others.

At the time of writing, PotreeCraft v2.1.2 is officially available in the QGIS plugin repository as an experimental plugin under the plugin ID #4836, and the codebase is freely available to access on my GitHub page[3].

3. QGIS PLUGIN DEVELOPMENT: PREPARATIONS

3.1 Setting up the development environment

Plugin development for QGIS is a fairly straightforward process, and generally does not require any sort of management of virtual environments, docker containers or any other special environments for the development flow to work. Generally speaking we need just a handful of things:

- QGIS: the software we're developing a plugin for.
- Python: the main language of development. (note: we plan to use the interpreter bundled with QGIS.)
- pdal: a pointcloud abstraction library, we'll use it during debugging and testing.
- pb_tool: a tool which will help us to compile, deploy and zip our plugin.
- Qt5 designer: a tool which we will use to edit the layout of the interface.
- Plugin reloader: a tool which re-initializes a freshly deployed plugin in QGIS.

I have experience on Windows and Linux-based plugin development, as the v.1.x.x series of the plugin was made on Windows 10, while the current 2.x.x version is primarily being developed on two laptops running Linux Mint, with minimal testing being done on a Windows 11 system, so the following sections will be divided accordingly.

3.1.1 Working on Windows

OSGeo4W

On windows systems it is highly recommended to install QGIS as part of the OSGeo4W package. The installer can be downloaded from the following link:

<https://www.osgeo.org/projects/osgeo4w/>

After getting through the usual questions of user privileges, installation folder locations and download mirror selection, the important part comes when the installer will ask the

user on what sort of packages he/she wishes to install as part of the OSGeo4W ecosystem. The installer has the packages sorted in four main categories:

- Command line utilities
- Desktop
- Libs
- Web

The user may select a variety of these for an even larger variety of reasons, but in this case the following are required:

Desktop category:

- qgis or any other version (like qgis-full)
- qt5 tools

Libs:

- pdal

We are free to select other items as well, but these are the ones this project relies on heavily. After pressing next, a prompt will appear which lists a few dozen dependencies we need to install also, which we have to agree to.

After the installation we will notice that if we try to use the Python interpreter just by simply trying to launch it, it will not start. If we use PowerShell or the old command line to launch it, we can read the error message to examine the problem.

Python fails very early during startup, before it can even initialize text encoding support. To do that initialization, CPython must import the standard-library package encodings (Figure 3.2). In a normal Python 3.12 installation, that module is normally found in `Python312/Libs/encodings`, instead it is looking for it in `OSGeo4W/bin/Lib/encodings`, and that path simply does not exist. In the OSGeo4W package the Python interpreter's core parts are scattered around in different folder structures around the project. In order to fix this, we need to set a series of environment variables to chain it all back together. The cleanest way to do this is to create a batch file, which does all this for us.

We need to look into our installation folder and adjust the template to match the Python version we're using, and the base installation directory[4].

```
Current thread 0x00008b2c (most recent call first):
  <no Python frame>
PS C:\Users\berest\OSGeo4W\bin> .\python.exe
Could not find platform independent libraries <prefix>
Python path configuration:
  PYTHONHOME = (not set)
  PYTHONPATH = (not set)
  program name = 'C:\Users\berest\OSGeo4W\bin\python.exe'
  isolated = 0
  environment = 1
  user site = 1
  safe_path = 0
  import site = 1
  is in build tree = 0
  stdlib dir = 'C:\Users\berest\OSGeo4W\bin\Lib'
  sys._base_executable = 'C:\\Users\\berest\\OSGeo4W\\bin\\python.exe'
  sys.base_prefix = 'C:\\Users\\berest\\OSGeo4W\\bin'
  sys.base_exec_prefix = 'C:\\Users\\berest\\OSGeo4W\\bin'
  sys.platlibdir = 'DLLs'
  sys.executable = 'C:\\Users\\berest\\OSGeo4W\\bin\\python.exe'
  sys.prefix = 'C:\\Users\\berest\\OSGeo4W\\bin'
  sys.exec_prefix = 'C:\\Users\\berest\\OSGeo4W\\bin'
  sys.path = [
    'C:\\Users\\berest\\OSGeo4W\\bin\\python312.zip',
    'C:\\Users\\berest\\OSGeo4W\\bin',
    'C:\\Users\\berest\\OSGeo4W\\bin\\Lib',
  ]
Fatal Python error: init_fs_encoding: failed to get the Python codec of the filesystem encoding
Python runtime state: core initialized
ModuleNotFoundError: No module named 'encodings'

Current thread 0x000062c4 (most recent call first):
  <no Python frame>
PS C:\Users\berest\OSGeo4W\bin> |
```

Figure 3.1: Python interpreter crashes on launch after OSGeo4W installation

```

@echo off
SET OSGEO4W_ROOT=C:\Users\berest\OSGeo4W
call "%OSGEO4W_ROOT%\bin\o4w_env.bat
@echo off
path %PATH%;%OSGEO4W_ROOT%\apps\qgis\bin
path %PATH%;%OSGEO4W_ROOT%\apps\Qt5\bin
path %PATH%;%OSGEO4W_ROOT%\apps\Python312\Scripts
::set PATH=%PATH%;C:\Program Files\7-Zip\
set PYTHONPATH=%PYTHONPATH%;%OSGEO4W_ROOT%\apps\qgis\python
set PYTHONHOME=%OSGEO4W_ROOT%\apps\Python312
set PATH=C:\Program Files\Git\bin;%PATH%
cmd.exe

```

Listing 3.1: Example batch file for OSGeo4W Python interpreter launch

After running the batch file we can try typing `python` command , and if inside the interpreter the line `import qgis` throws no errors, we can be certain that we have a working instance of a PyQGIS development environment. The next step would be to make a copy if this batch file, and modify it slightly so it would not launch a command line terminal, but instead our IDE of choice, such as Visual Studio Code.

```

@echo off
SET OSGEO4W_ROOT=C:\Users\berest\OSGeo4W
call "%OSGEO4W_ROOT%\bin\o4w_env.bat
@echo off
path %PATH%;%OSGEO4W_ROOT%\apps\qgis\bin
path %PATH%;%OSGEO4W_ROOT%\apps\Qt5\bin
path %PATH%;%OSGEO4W_ROOT%\apps\Python312\Scripts
set PYTHONPATH=%PYTHONPATH%;%OSGEO4W_ROOT%\apps\qgis\python
set PYTHONHOME=%OSGEO4W_ROOT%\apps\Python312
::set PATH=%PATH%;C:\Program Files\7-Zip\
set PATH=C:\Program Files\Git\bin;%PATH%
start "VS Code aware of QGIS" /B "C:\Program Files\Microsoft VS Code\Code.exe" %*

```

Listing 3.2: Batch file for launching VS Code using the OSGeo4W Python interpreter

If IntelliSense in VS Code provides suggestions for the `qgis` module, it indicates that the Python language server can locate and analyze the package.

Note that in both instances there's a line for 7-Zip commented out – this is an optional tool, which we only need if we plan to export zip files with `pb_tool` at the means of distributing the plugin during the development process. We'll come back to this point when we're discussing `pb_tool` and the development loop itself.

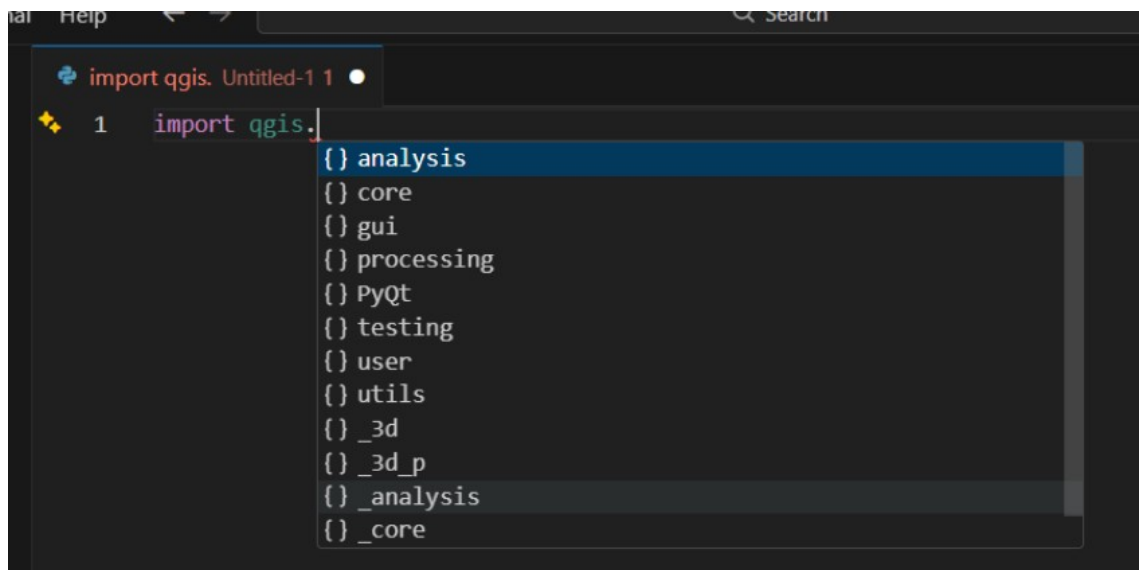


Figure 3.2: Testing if IntelliSense is capable of reading the `qgis` package

User installed plugins in QGIS are located in the `QGISprofiles\default\python\plugins` folder. We can install a plugin through QGIS's manual zip install of course, or copy our code manually in here, but it is not advised. There is a way better method to ship our code to QGIS for testing, that is why we need the Plugin builder tool[5].

pb_tool

The `pb_tool` installs as a python package to whichever python installation we wish to install it to, just run the command in Figure 3.3.

```
# Installing pb_tool (default settings)
pip install pb_tool
```

Figure 3.3: Installation command for pb_tool

Following the installation type “pb_tool” to check if the installation went correctly. If pb_tool is not recognized as a command, it is likely that the Python path is missing from the system / user’s environment variables. Make sure to add the base Python folder and the Scripts subfolder as well.

Since we selected “qt5-tools” during the OSGeo4W installation, we should already have a start menu entry for Qt5 Designer (Figure 3.4). Similarly with our Python interpreter, this is also launched by a batch file, but this one is pre-made, so we do not have to worry about editing anything here. Qt5 Designer’s executable lives inside the the OSGeo4/apps/qt5/bin/designer.exe location. The batch file sets two environment variables so the custom QGIS widgets would be available for it, so the shortcut points to qgis-designer.bat by default.

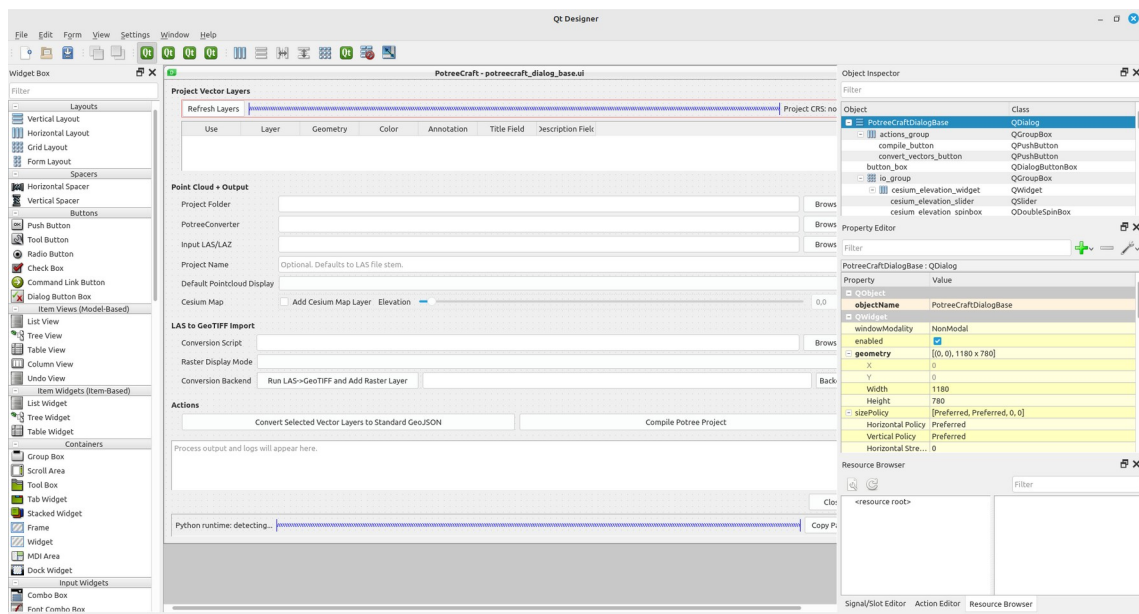


Figure 3.4: Qt5 designer

3.1.2 Working on Linux

Linux by nature is a much more modular and customizable environment compared to windows. Setting up the development environment in theory should take even fewer commands and less workarounds, but reality often puts a few obstacles in our way. This is one of those cases. The setup discussed here was made and tested on Linux Mint, specifically the version 21.3 and 22.1 releases.

QGIS

Let's start by installing our main software, QGIS. Since we want to integrate QGIS into our development environment, for simplicity's sake I'd recommend staying away from the flatpak version, we want QGIS as a heap of system packages (Figure 3.5).

```
# First we need to download and add the keyring:
sudo mkdir -p /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/qgis-archive-keyring.gpg
https://download.qgis.org/downloads/qgis-archive-keyring.gpg
# Add the repository:
echo "deb [signed-by=/etc/apt/keyrings/qgis-archive-keyring.gpg]
https://qgis.org/ubuntu $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/qgis.list
# Run update so changes would apply, and we can install the packages
sudo apt update
# QGIS is interestingly split up to several packages which we might need.
sudo apt install qgis python-qgis qgis-plugin-grass
```

Figure 3.5: QGIS installation commands on a linux system

GRASS is a library for advanced spatial analysis and it is not a strict requirement, as it was seen in the windows example I did not list it there as a strict requirement – the reason why i left it as part of this command is that unlike in windows, installation is done by console commands only, and i wanted to illustrate the modular nature of QGIS here.

Qt5

The next item should be PyQt5 tools. Without this pb_tool won't be able to compile any plugins.

```
# Installing Qt packages and devtools  
sudo apt install python3-pyqt5 pyqt5-dev-tools qttools5-dev-tools
```

Figure 3.6: devtools and Qt package installation commands on a linux system

The python3-pyqt5 is the core package, allowing Python to access core modules like QtWidgets or QtCore, while pyqt5-dev-tools is the package allowing the conversion of .ui files to python code or the use of qt resource files (.qrc). And finally qttools5-dev-tools package installs the Qt designer on Linux (Figure 3.6).

pdal

pdal (Point Data Abstraction Library) is an open-source library and command-line tool for processing, analyzing, and transforming point cloud data (such as LiDAR) through flexible pipelines. I did not mention this during the Windows section, but we'll use this library to examine on our pointcloud files later on. Here we need to install it with the following command:

```
# Installing pdal (point data abstraction library)  
sudo apt install pdal
```

Figure 3.7: pdal installation commands on a linux system

pb_tool

The Plugin builder tool is not managed through apt, so we have to install it the same way as we did it on Windows.

In some cases there might be a problem with Qt/PyQt compatibility, which manifests in form of a core dump while trying to deploy packages. Generally it is advised to manage Qt installations for this project through the apt package manager. We can be extra safe and install it as a user module (Figure 3.8).

```
# Installing pb_tool (default settings)
pip install pb_tool

# Installing pb_tool as a user module
python3 -m pip install --user pb_tool
```

Figure 3.8: pb_tool installation commands

Using `--user` flag will install `pb_tool` into the user’s site packages folder, forcing Qt to rely on the previously system-installed Qt packages, preventing the possibility of a version mismatch. The key takeaway is to avoid installing Qt with `pip install PyQt5` at all costs.

3.1.3 Getting the external dependencies

Plugin reloader

The final piece of the puzzle is a plugin we need to install in QGIS, called “Plugin reloader”. This can be done in Plugins → Manage and install plugins → All, and there search for “Plugin reloader”. After installing it, we’ll get a button which can be used to reload the code in our plugin with a click of a button, instead of restarting QGIS itself after every deployment. If we’re using this for the first time, or we do not have our target plugin installed yet, the dropdown menu can be used to select the plugin we’re going to force-reload with the main button (Reload plugin: [name of the plugin], as it can be seen in Figure 3.9).

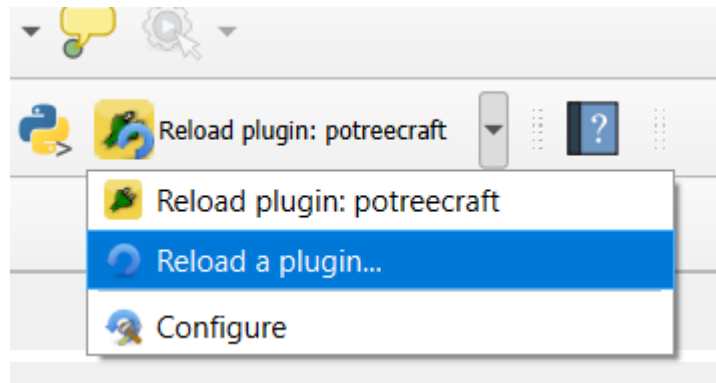


Figure 3.9: Plugin reloader

LAStools (blast2dem)

This is an optional step, and in the current stage of the development I'd say only applicable for Windows users. LAStools is (unsurprisingly) a collection of binaries for working with pointclouds. One of these binaries included in the package is called "blast2dem", which can be used to create 2D raster images of pointclouds. It has rich settings through parameter inputs, and generally it was my go-to solution to visualize the location of the pointcloud in the 1.x.x. versions of this plugin. In version 2.x.x I am moving away from this due to the .exe nature of the precompiled binaries, and brought in my own implementation to solve the same problems using the rasterio library, however I have to say blast2dem is probably superior in many aspects still (which in the 2.1.2 version of the plugin cannot be seen, due to the lack of implementation of using said input parameters).

The files can be accessed from this URL:

<https://lastools.github.io/>

PotreeConverter

This tool is the beating heart of this project, the whole idea is to extend the capabilities of the PotreeConverter tool by providing a GUI and the possibility to integrate vectors into the scenes made with it.

As of writing this, PotreeConverter 2.1.1 is the latest release, and the plugin discussed in this thesis was tailored to support that version, and that version only. This may change in later versions of the PotreeCraft plugin of course, but for now, make sure you download a Windows or Linux precompiled binary provided here. If you are on linux, make sure to add execution privileges to the binary with `chmod +x` as well.

<https://github.com/potree/PotreeConverter>

The detailed use of PotreeConverter and blast2dem will be discussed in later parts of this document.

Starting up a webserver

Due to the web-based nature of potree, we must have some sort of locally running web-server in order to use what we generated. Most modern programming languages ship some sort of basic solution for this problem. In our case, the standard windows install binary contains the `http.server` module. On linux, on minimal systems this module might be missing from the default installation package, but if we run into this problem, installing `python3-full` instead of `python3` would solve this under most circumstances.

If we open a command line tool in the project directory, and then type `python3 -m http.server`, our project's content will be accessible in the browser on the localhost or 127.0.0.1 address.

There are other options as well, such as `npm`'s `http-server` or `serve`, or the Webpack Dev Server, Parcel or Apache Tomcat, just to name a few.

4. QGIS PLUGIN DEVELOPMENT: BUILDING THE PLUGIN

Now that we have all the tools required for this project, we can take a closer look on how this plugin was created. First, I will provide a general workflow of the development process, following that I will go through the plugin on a module-by-module basis, describing the role it possesses in the whole architecture. Since this is an open source project running GNU General Public License version 2 or later, the full codebase is available on Github. Due to length constraints I will only highlight key parts of the algorithm itself, and use line references wherever possible. An extract of the codebase will be attached in the appendix.

4.1 The Development loop

The development procedure I've done here can be described as a localized, simplified version of CI/CD.

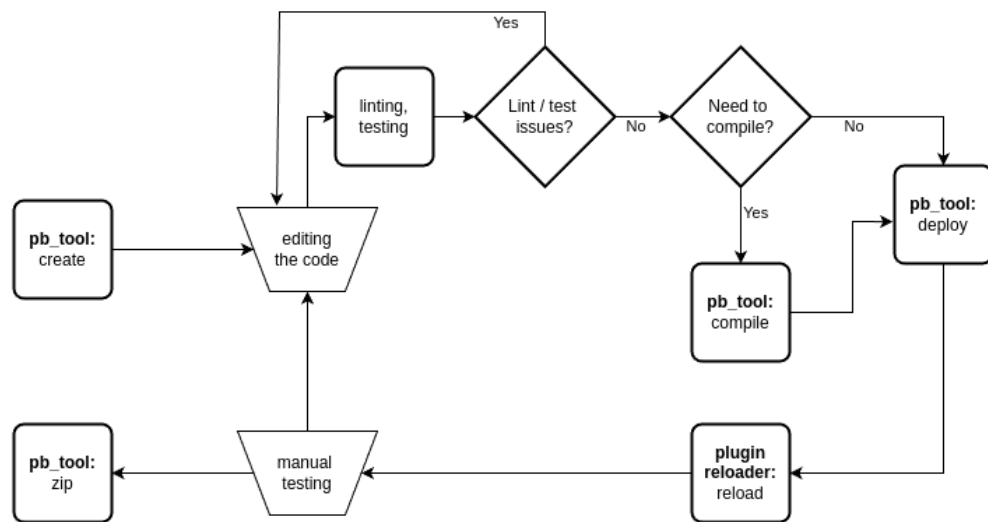


Figure 4.1: The development loop

CI/CD stands for Continuous Integration / Continuous Development[6], which is a development practice where changes in the codebase trigger an automated validation, build and delivery procedure. The point of CI/CD is that the above mentioned processes require no input from the developer team and is done through automated system.

What is missing here is the full automation of this development cycle, while we're not copying the code by hand, we'll still have to call `pb_tool deploy` and press the reload button ourselves – otherwise our software is built through incremental changes.

4.1.1 Step 1 – initialize the module with `pb_tool create`

```
user@linuxmintpc:~/Documents/test_plugin$ pb_tool create
Module name (lowercase): test_qgis_plugin
Class name: (mixed case): TestQgisPlugin
Text for the plugin menu: Test QGIS Plugin
Creating TestQgisPlugin in module test_qgis_plugin with menu
text Test QGIS Plugin
Using /home/user/.gitconfig to set some items in the plugin
Created new config file in pb_tool.cfg
user@linuxmintpc:~/Documents/test_plugin$ tree
.
├── icon.png
├── __init__.py
├── metadata.txt
├── pb_tool.cfg
├── readme.txt
├── resources.qrc
├── results.html
├── test_qgis_plugin_dialog_base.ui
├── test_qgis_plugin_dialog.py
└── test_qgis_plugin.py
```

0 directories, 10 files

Listing 4.1: pb_tool create output

Running the `pb_tool create` command is going to initialize a plugin. The command will ask the user to provide a few answers regarding the naming of the module. Through this process, the following files were created:

- **icon.png** – a 23 pixels in width, 24 pixels in height stock plugin icon.
- **__init__.py** – Functionally this file contains an `classFactory` function definition importing the main class of our plugin. It also has the user's name and email with some short license statement about GNU GPL v2.
- **metadata.txt** – this file contains the metadata for the plugin, such as the email of the developer, homepage of the project, etc. The contents of this file will appear in QGIS as information about the plugin if we select it on the plugin management screen.
- **pb_tool.cfg** – this file contains information for the plugin builder itself on what are the elements of this plugin, it relies on what's in this file when we call `pb_tool deploy` for example.
- **readme.txt** – a short set of tips on what the developer should do after he ran the `create` command and finds this file.
- **resources.qrc** – an XML based file containing information on the Qt5 resources (the non-python files) our plugin uses, for example the icon file we've mentioned earlier. `pb_tool's` `compile` command looks into this file for example.
- **results.html** – another information page for new developers informing them on what happened after running `pb_tool create`, and what sort of steps they can take to continue their work effectively.
- **[name of the plugin]_dialog_base.ui** – an XML based description of the GUI window's widget positions. This is the file we can edit with Qt designer.
- **[name of the plugin]_dialog.py** – this file starts it's life relatively empty, but it will be probably the one seeing the most growth in size. Every button's functional implementation (or at least the calls to the implementations) will reside here, as this is the place where the interface and the underlying functionality is going to be connected.
- **[name of the plugin].py** – this file contains functions defining on what code to run in certain phases of events of the plugin's lifecycle, such as "run" or "init-

Gui”. The code we see there is responsible for managing the basic functionality and correct integration with QGIS itself.

4.1.2 Step 2 – Editing the code

This is probably the most straightforward part of the task, we make changes in the code, add features, modify the GUI, etc.

4.1.3 Step 3 – linting and testing

After making some changes it is advised to run the project through some linting checks and tests for example. When it comes to Python software development PEP 8 is the main style guideline most linters try to enforce[7]. It defines conventions for things like indentation, line length, naming, blank lines, imports, and spacing so Python code stays consistent and easier to read. To enforce such conventions developers often add a linting tool to their arsenal which checks the files if they adhere to these standards, and warns the developer about them. There’s a whole arsenal of popular linting tools for this language - pylint, pycodestyle, flake8 or ruff, just to mention a few. In this project I’m using ruff, which is one of the more easy-to-use linting tools for python.

```
# Installing ruff linter
pip install ruff

# Usage
ruff check [folder_path]
```

Figure 4.2: installation command and basic usage of ruff

Installing and running can be done with commands seen in Figure 4.2. If ruff finds anything not up to the PEP 8 coding standard, we’ll get an output like in Listing 4.2. Important to note, the code runs even with these in it.

```

user@linuxmintpc:~/Documents/GitKraken/PotreeCraft$ make lint
ruff check qgis_plugin tests
F403 `from .resources import *` used; unable to detect undefined names
--> qgis_plugin/potreecraft.py:25:1
24 | # Initialize Qt resources from file resources.py
25 | from .resources import *
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
26 | # Import the code for the dialog
27 | from .potreecraft_dialog import PotreeCraftDialog

F841 Local variable `blast2dem_path` is assigned to but never used
--> qgis_plugin/potreecraft_dialog.py:886:17
884 |             self._show_warning(f"blast2dem executable not found:\n{blast2dem_executable}")
885 |             return
886 |             blast2dem_path = str(blast2dem_executable)
   |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
887 |
888 |             legacy_flag = LEGACY_RASTER_MODE_FLAGS.get(mode)
help: Remove assignment to unused variable `blast2dem_path`

Found 2 errors.
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option).
make: *** [Makefile:15: lint] Error 1

```

Listing 4.2: ruff linting error example

Tests have a bit higher importance in my opinion. While a module might look “sub-standard” without adhering to linting rules, if we skip thorough testing real problems can arise. To catch common problems before they reach the manual testing phase, developers often use automated tests in their projects. Some development workflows are built around writing tests or behavior specifications early in the process, such as TDD (Test-Driven Development) and BDD (Behavior-Driven Development). For running tests I’m using pytest and its QGIS extension, pytest-qgis. These can be installed with the following command, as seen in Figure 4.3.

```

# Installing testing tools
pip install pytest pytest-qgis

```

Figure 4.3: installation command for testing tools

Pytest discovers tests automatically based on naming conventions, typically from files such as `test_*.py` or `*_test.py`, often placed in a `tests` directory. It then runs the test functions it finds, usually those named `test_*`, and reports which ones passed or failed.

4.1.4 Step 4 – Compiling the plugin

Compilation of resources is a necessary step after editing the `.ui` files or modifying the resource `.qrc` resource file (for example changing the icon). This process converts the language-agnostic Qt interface’s XML data format located in the plugin’s `.ui` files into python code. Normally this step is included in deployment as well, unless we do quick deployment. Compilation can be called by using the following command, as seen in Figure 4.4.

```
# Compilation without deployment
pb_tool compile
```

Figure 4.4: plugin compilation command

4.1.5 Step 5 – Deploying the plugin

While compilation could be a step with skip in most cycles, deployment is mandatory if we’d like to see our code in action. Deployment of a plugin consists of the following steps:

- removal of the old version of the plugin from QGIS from it’s location:
`QGIS3/profiles/default/python/plugins/[name_of_the_plugin]/`
- compilation, `pb_tool` silently runs the `compile` command to get the possible changes in resources ready (unless we use `--quick` flag.)
- copying the new version of the plugin back into the same location mentioned above.
- optionally `pb_tool` can also build help documentation and translation files.

The command for deployment is the following, as seen in Figure 4.5.

```
# Deploying the plugin
pb_tool deploy
```

Figure 4.5: plugin deployment command

4.1.6 Step 6 – Reloading the plugin

This step is for the majority of the development process is just a click of a button, however the importance of it cannot be understated. After deploying a new version of our plugin, if QGIS is open, the changes will not take effect until the plugin is reloaded. (Without the reloader the simplest way to achieve this is to restart QGIS itself.)

To do this, we simply just have to press the button shown in Figure 3.9.

4.1.7 Step 7 – Testing the changes

This step involves the manual testing of our plugin. We open the plugin, and try to use the feature we’ve modified in our recent changes. This can be anything based on what our plugin’s functionality.

4.1.8 Step 8 – Shipping the plugin with pb_tool zip

In order to install a plugin on any given QGIS copy (which is not our development environment), we have two options: first, is to select an approved and tested plugin from the official QGIS plugin repository, and the second is to install a plugin directly from a .zip file. In order to quickly package our plugin for distribution pb_tool includes a zip command.

In case we install pb_tool on a Windows machine, it is necessary to install a version of 7-zip, if we’d like to use this feature, plus fill out and enable the line pointing to its location in the OSGeo4W batch files we’ve seen in Listing 3.1 or Listing 3.2, depending on where do we call pb_tool from (separate cmd or PowerShell window, or the terminal integrated into VS Code).

Furthermore if the developer goes through the registration process, and submits a plugin for the official repository, this also has to be done by uploading a copy of the plugin in a .zip format.

Zippping with pb_tool can be done by using the following command seen in Figure 4.6.

```
# Packaging plugin for distribution  
pb_tool zip
```

Figure 4.6: command for packaging the plugin with pb_tool

4.1.9 Getting everything ready in one place

Since the development process often requires the developer to run the above mentioned commands in different folders, it is customary to gather our commands into a Makefile, and assign them short command aliases, so we do not have to type out long commands each time we plan to run a series of different tests, or add a series of parameters after an executable.

In Listing 4.3 we can see the example of a Makefile where tests, linting commands and pb_tool's commands are all collected into one place, so we can have a terminal open in this folder and just type `make test` or `make deploy` without having to change the working directory for any reason.

```

PYTEST ?= pytest
RUFF ?= ruff
PB_TOOL ?= pb_tool
PLUGIN_DIR ?= qgis_plugin

.PHONY: help lint lint-plugin lint-tests compile deploy zip unit-test qgis-dialog-test test

help:
@echo "make lint Run linting for the whole plugin and tests"
@echo "make lint-plugin Run linting for the full qgis_plugin package"
@echo "make lint-tests Run linting for the tests package"
@echo "make compile Run pb_tool compile in qgis_plugin"
@echo "make deploy Run pb_tool deploy in qgis_plugin"
@echo "make zip Run pb_tool zip in qgis_plugin"
@echo "make unit-test Run plain pytest tests that do not need QGIS fixtures"
@echo "make qgis-dialog-test Run the pytest-qgis dialog smoke test"
@echo "make test Run the full starter test suite"

lint:
$(RUFF) check qgis_plugin tests

lint-plugin:
$(RUFF) check qgis_plugin

lint-tests:
$(RUFF) check tests

compile:
cd $(PLUGIN_DIR) && $(PB_TOOL) compile

deploy:
cd $(PLUGIN_DIR) && $(PB_TOOL) deploy

zip:
cd $(PLUGIN_DIR) && $(PB_TOOL) zip

unit-test:
$(PYTEST) -q tests/test_plugin_factory.py

```

Listing 4.3: the Makefile of the PotreeCraft project.

4.2 Architectural overview of the PotreeCraft project

The following diagram offers a quick look at the relations between files or components found in the project. The following section will discuss elements of this diagram.

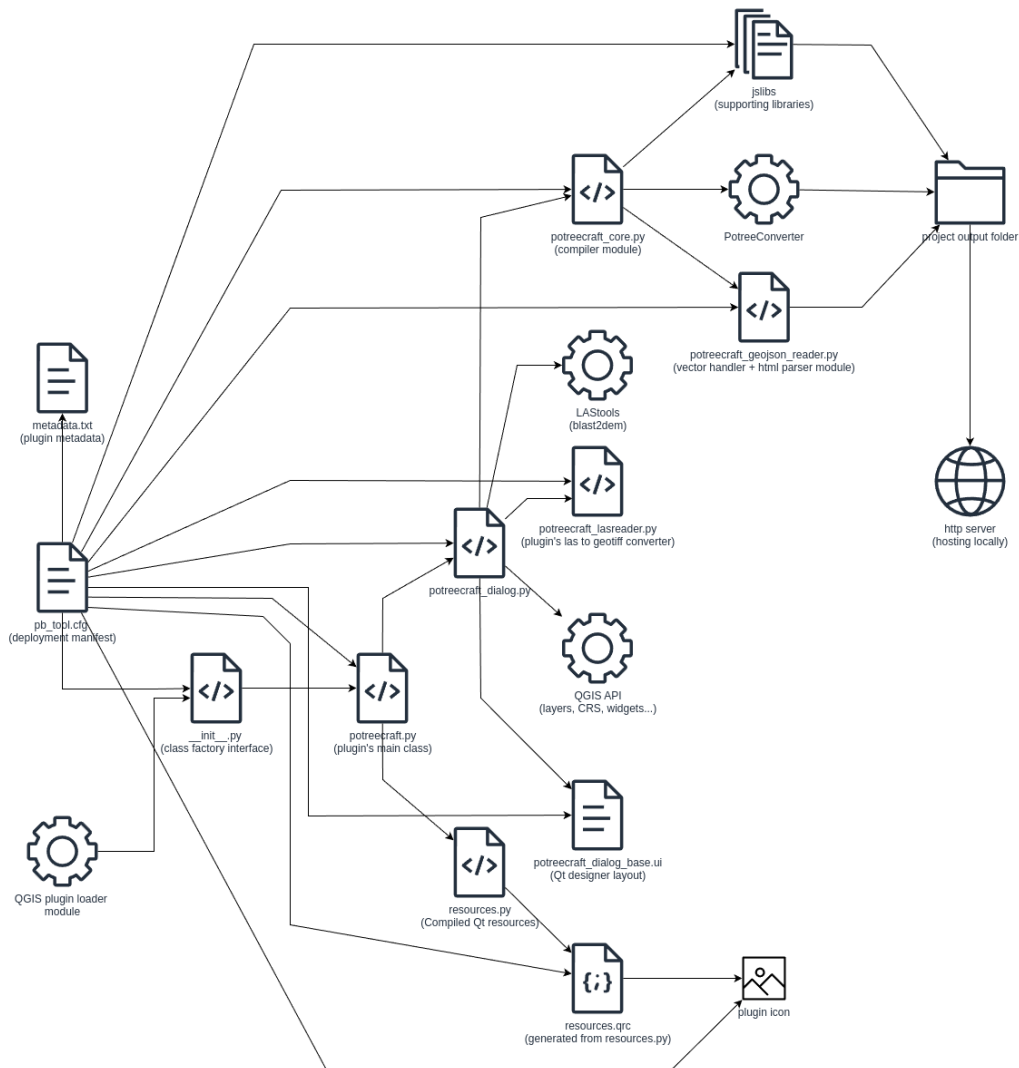


Figure 4.7: High level overview of the plugin

Figure 4.7 contains the reference structure of the plugins' files. We'll go over the individual parts one by one below.

4.2.1 QGIS Plugin loader

The QGIS plugin loader is the entry mechanism that lets QGIS discover and instantiate PotreeCraft. In practice, QGIS reads the plugin package, evaluates its metadata, imports `__init__.py`, and calls `classFactory(iface)`. That factory function returns the main PotreeCraft object, which then registers the toolbar button and menu entry used to open the user interface.

4.2.2 pb_tool.cfg

The `pb_tool.cfg` is the deployment configuration file used by `pb_tool`. In PotreeCraft it lists the Python source files, the main `.ui` file, the Qt resource collection, the plugin icon, metadata, license file, and the `jslibs` directory that must be copied into the distributable plugin bundle. This file is not part of runtime execution, but it is important in the build and release workflow because it defines what gets shipped.

4.2.3 metadata.txt

The `metadata.txt` is the manifest that describes the plugin to QGIS. It contains the plugin name, version, supported QGIS range, author information, repository URLs, category, icon reference, and summary text shown inside the QGIS Plugin Manager. The file therefore acts as the public identity card of PotreeCraft and is required for installation and distribution.

4.2.4 __init__.py

The `__init__.py` is the Python entry point expected by QGIS. Its main responsibility is minimal by design: it exposes `classFactory(iface)`, imports the `PotreeCraft` class, and returns a plugin instance bound to the live `QgsInterface`. This separation keeps the bootstrap layer thin and delegates all application logic to the main plugin class.

4.2.5 potreecraft.py

`potreecraft.py` contains the main plugin controller class. Its responsibility is to bridge QGIS and the PotreeCraft dialog by initializing translations, creating menu and toolbar actions, lazily instantiating the dialog, and handling plugin load and unload events.

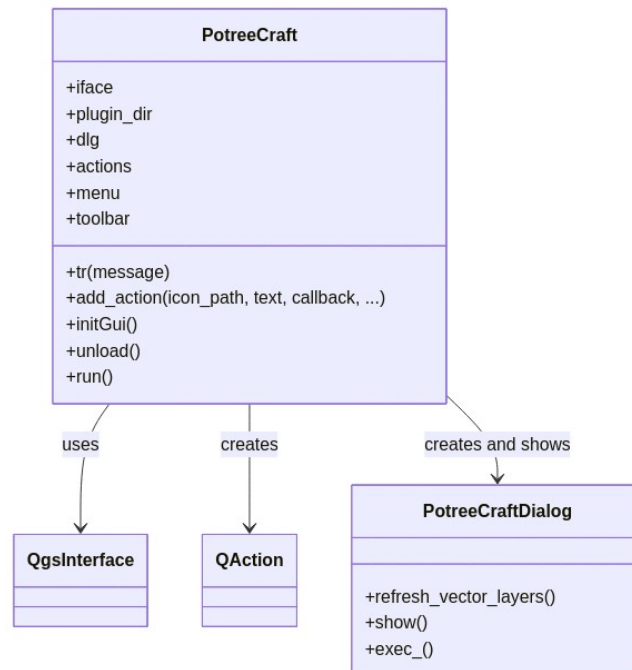


Figure 4.8: Class diagram for *potreecraft.py*

Architecturally, this module is the shell around the user interface: it does not perform conversion itself, but it controls when the dialog becomes available and how the plugin integrates into QGIS. (For detailed method descriptions of `PotreeCraft` and `PotreeCraftDialog` classes see appendix chapter A.I.I and A.I.II.)

4.2.6 `resources.py`

`resources.py` is the auto-generated Python module produced by the Qt resource compiler. It embeds binary resource data and registers it with Qt so that assets can be referenced through resource paths such as the icon file bundled in the plugin package. Since this file is automatically generated code, and it contains no meaningful domain classes, a UML class diagram is not especially useful here.

4.2.7 `resources.qrc`

The `resources.qrc` is the source resource collection file used by Qt. During build time this XML file is compiled into `resources.py`, which is then imported by the main plugin module.

4.2.8 plugin icon file

The plugin icon file (`pc_icon_24.png`), provides the visual identifier that appears in the QGIS toolbar, menu, and plugin manager. Even though it is only a small asset, it is operationally important because both `metadata.txt` and the Qt resource system refer to it. In other words, it is part of the plugin's usability and discoverability rather than its business logic. By default the icon is a .png file which is 24×24 pixels in size.

4.2.9 potreecraft_dialog.py

The `potreecraft_dialog.py` is the main application module of the plugin. It defines the dialog class shown to the user and also the worker classes responsible for long-running raster conversion and project compilation tasks. The module coordinates UI state, QGIS layer inspection, GeoJSON export, manifest writing, external tool invocation, and background-thread execution, so it serves as the orchestration layer of the whole plugin. (For detailed method descriptions see appendix chapter A.I.II).

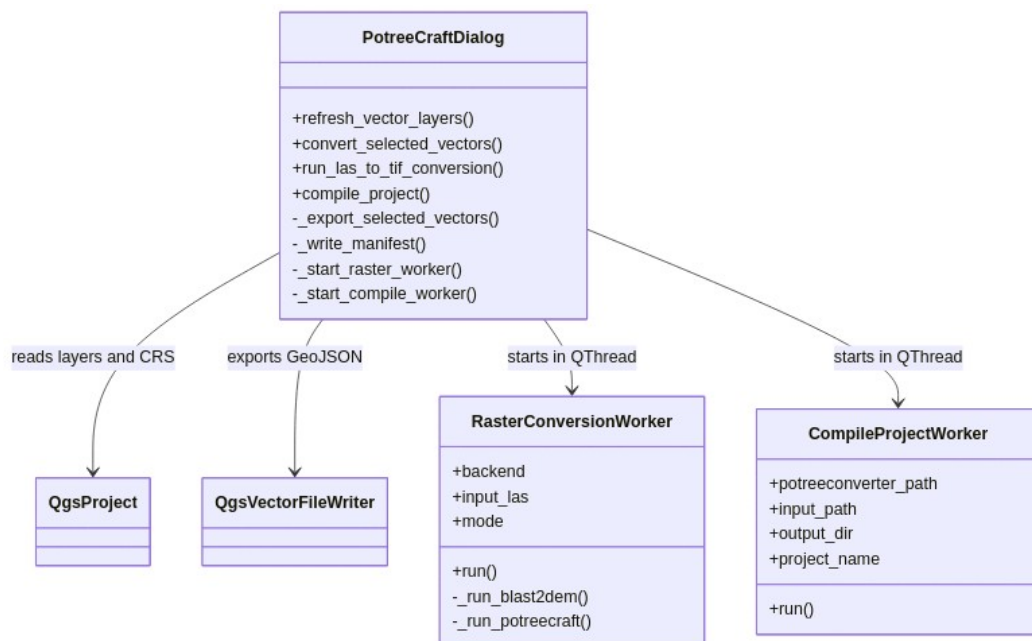


Figure 4.9: Class diagram for `potreecraft_dialog.py`

4.2.10 `potreecraft_dialog_base.ui`

`potreecraft_dialog_base.ui` is the Qt Designer form definition for the plugin window. It describes the static widget tree, including the vector-layer table, file path inputs, PotreeConverter controls, raster conversion controls, Cesium options, logging area, and action buttons. The `.ui` file therefore captures the structure of the interface, while `potreecraft_dialog.py` supplies the interactive behavior.

4.2.11 `potreecraft_core.py`

The `potreecraft_core.py` contains the procedural backend for Potree project compilation. Its functions validate and normalize paths, execute PotreeConverter, copy the required runtime JavaScript libraries, synchronize exported GeoJSON files into the final output structure, resolve the generated point-cloud folder name, and finally call the HTML generator. Because the module is function-based rather than class-based, a UML class diagram is not particularly applicable. (For detailed method descriptions see appendix chapter A.I.V.)

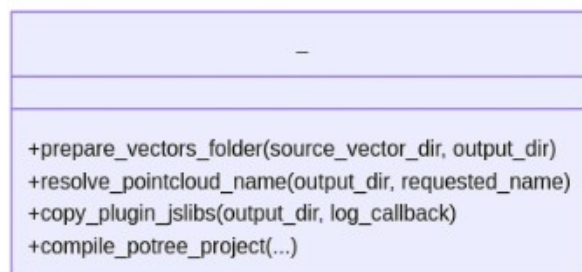


Figure 4.10: Class diagram for `potreecraft_core.py`

4.2.12 `potreecraft_lasreader.py`

`potreecraft_lasreader.py` implements the built-in LAS or LAZ to GeoTIFF conversion backend. The central `FlatLas` class reads point-cloud coordinates and intensity values with `laspy`, interpolates them onto a regular raster grid using `numpy`, and writes the resulting GeoTIFF with `rasterio`. This module is especially important

because it provides a portable internal alternative to the older external `blast2dem` workflow. (For detailed method descriptions see appendix chapter A.I.IV.)

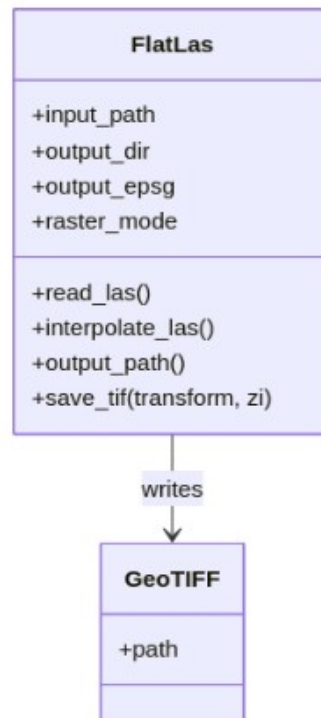


Figure 4.11: Class diagram for `potreecraft_lasreader.py`

4.2.13 LAStools: `blast2dem`

The `blast2dem` binary from LAStools is the legacy external rasterization backend supported by the dialog for compatibility reasons. When this mode is selected, PotreeCraft launches the executable as a subprocess and passes arguments that control the input file, output TIFF path, verbosity, and the selected raster display mode. This tool has a higher level of customizability when it comes to settings, but the current version of the code-base only provides this as an option on Windows machines.

4.2.14 jslibs folder

The jslibs folder stores the client-side runtime libraries that PotreeCraft copies into generated projects. In the current repository this includes `Cesium` version 1.83 and a project-specific `three.js` runtime tree (revision 109, instead of what comes bundled with our projekt when we use PotreeConverter 2.1.1). These files are not used inside QGIS itself; instead, they are deployed into the output web project so that ``potree_main.html`` can load the required JavaScript and CSS assets directly from the generated folder structure.

4.2.15 PotreeConverter

PotreeConverter is the main external processing dependency of the plugin. PotreeCraft does not replace it; instead, it wraps it in a more accessible QGIS-driven workflow. During compilation, the plugin calls the user-selected PotreeConverter executable, waits for the point-cloud web scene to be generated, then augments that output with vectors, runtime libraries, and a custom HTML front end. PotreeConverter can therefore be described as the foundation of the generated Potree scene, while PotreeCraft adds geospatial integration and presentation logic around it.

4.2.16 QGIS API

The QGIS API is the host platform interface used throughout the plugin. PotreeCraft depends on `QgsInterface` for UI integration, `QgsProject` for reading the active project state and CRS, `QgsVectorFileWriter` for GeoJSON export, `QgsMessageLog` for logging, and several PyQt classes for widgets, signals, threads, and dialogs. Without these APIs the plugin would lose both its GIS context and its native integration into the QGIS desktop environment.

4.2.17 project output folder

The project output folder is the generated web project root selected by the user in the dialog. This directory contains the PotreeConverter output, a `libs/` directory with copied runtime assets, a `vectors/` directory with exported GeoJSON files, the generated

potree_main.html, and the potreecraft_project_manifest.json metadata file. It acts as the handoff point between desktop authoring in QGIS and browser-based visualization.

4.2.18 http server

The HTTP server is not part of the plugin codebase itself, but it is a practical requirement for viewing the generated Potree project correctly in a browser. For simplicity's sake I recommend serving the output directory either with Python's built-in `http.server` module or with the `http-server` npm package, which I'm going to discuss later in chapter 3.1.3: Getting the external dependencies and 5.1.3: Putting the project together.

```
1 user@linuxmintpc:~/vm share/roadside test data no ces$ python3 -m http.server 8095
2 Serving HTTP on 0.0.0.0 port 8095 (http://0.0.0.0:8095/) ...
3 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /potree_main.html HTTP/1.1" 200 -
4 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/potree/potree.css HTTP/1.1" 200 -
5 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/openlayers3/ol.css HTTP/1.1" 200 -
6 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/spectrum/spectrum.css HTTP/1.1" 200 -
7 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/spectrum/spectrum.js HTTP/1.1" 200 -
8 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/jquery-ui/jquery-ui.min.css HTTP/1.1" 200 -
9 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/jstree/themes/mixed/style.css HTTP/1.1" 200 -
10 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/jquery/jquery-3.1.1.min.js HTTP/1.1" 200 -
11 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/jquery-ui/jquery-ui.min.js HTTP/1.1" 200 -
12 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/other/BinaryHeap.js HTTP/1.1" 200 -
13 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/tween/tween.min.js HTTP/1.1" 200 -
14 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/d3/d3.js HTTP/1.1" 200 -
15 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/proj4/proj4.js HTTP/1.1" 200 -
16 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/openlayers3/ol.js HTTP/1.1" 200 -
17 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/il8next/il8next.js HTTP/1.1" 200 -
18 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/jstree/jstree.js HTTP/1.1" 200 -
19 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/potree/potree.js HTTP/1.1" 200 -
20 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/plasio/js/laslaz.js HTTP/1.1" 200 -
21 127.0.0.1 - - [09/Apr/2026 14:18:04] "GET /libs/three.js_potreecraft/build/three.min.older.js HTTP/1.1" 200 -
```

Figure 4.12: Log file snippet of the python `http.server` module

4.3 Interaction between the modules

The following sequence diagrams will demonstrate the steps of interaction between the modules from the perspective of a single workflow. I will showcase the three main primary ones: vector export, raster import, and project compilation.

4.3.1 Raster import sequence

The raster-import sequence, as seen in Figure 4.13, shows how the plugin converts a .las/.laz file into a GeoTIFF and then loads it back into QGIS. When the user starts raster conversion, the dialog validates the input paths and selected mode, then launches `RasterConversionWorker` on a separate thread so the UI stays responsive. Depending on the chosen backend, the worker either calls the built-in `potreecraft_lasreader.py` conversion path or runs the external `blast2dem` executable; after a successful result, the dialog adds the produced raster file as a new QGIS raster layer and reports success to the user.

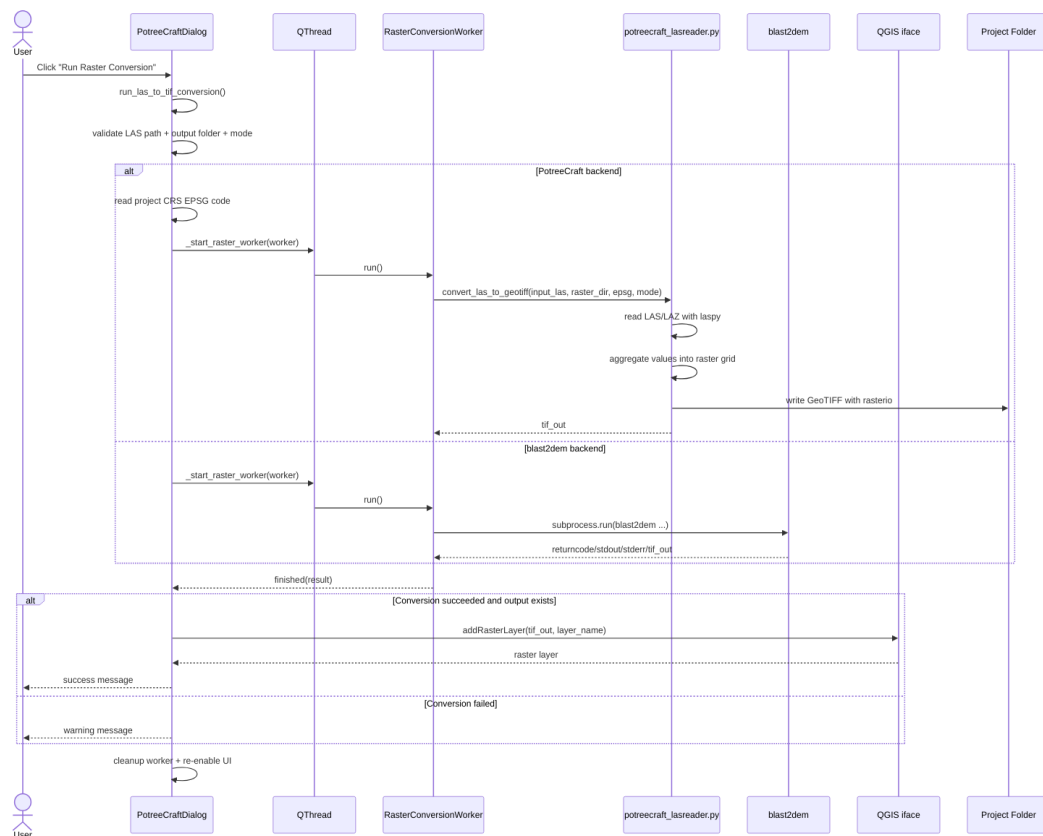


Figure 4.13: Raster import sequence diagram

4.3.2 Vector export sequence

Figure 4.14 shows the vector export workflow that starts when the user clicks 'Convert Selected Vector Layers' in the PotreeCraft dialog. The dialog validates the output paths,

collects the currently selected QGIS vector layers, prepares the `vectors/cache/` folder, and removes outdated GeoJSON exports. It then loops through each selected layer, exports it through `QgsVectorFileWriter`, and, on success, reopens the GeoJSON to embed PotreeCraft-specific color metadata. After all layers are processed, the dialog writes the project manifest and reports the result back to the user either as a success message or as a warning listing any failed layer exports.

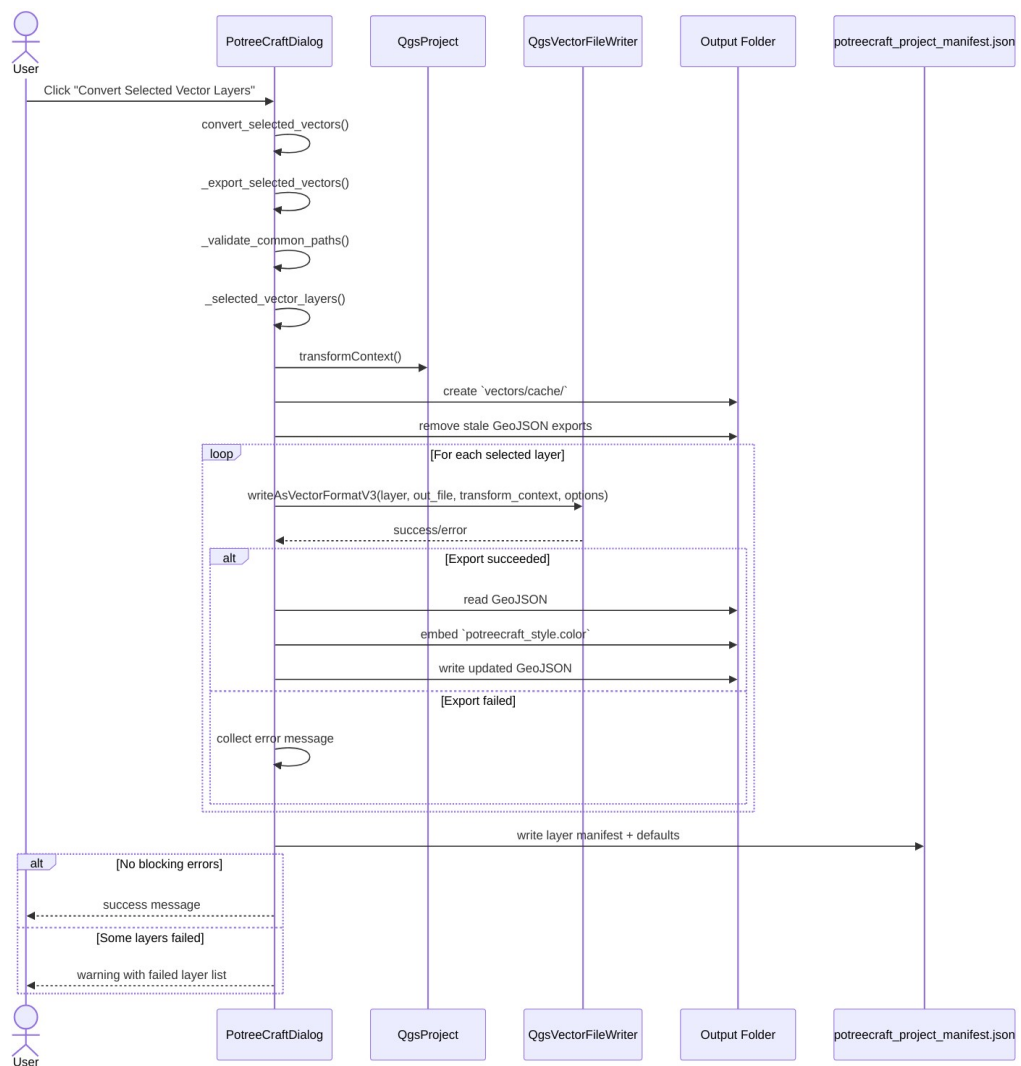


Figure 4.14: Vector export sequence diagram

4.3.3 Compile project sequence

Figure 4.15 illustrates the compile-project sequence, which describes how the plugin turns a prepared QGIS project into a complete Potree web output. After the user clicks Compile, PotreeCraftDialog validates the selected .las/.laz file, output folder, and PotreeConverter path, then performs vector export before handing the heavy work to CompileProjectWorker on a background thread. That worker calls `compile_potree_project(...)` in `potreecraft_core.py`, which runs PotreeConverter, checks the generated output, copies the required runtime libraries, moves the exported GeoJSON overlays into the project, resolves the point-cloud name, and finally generates `potree_main.html`.

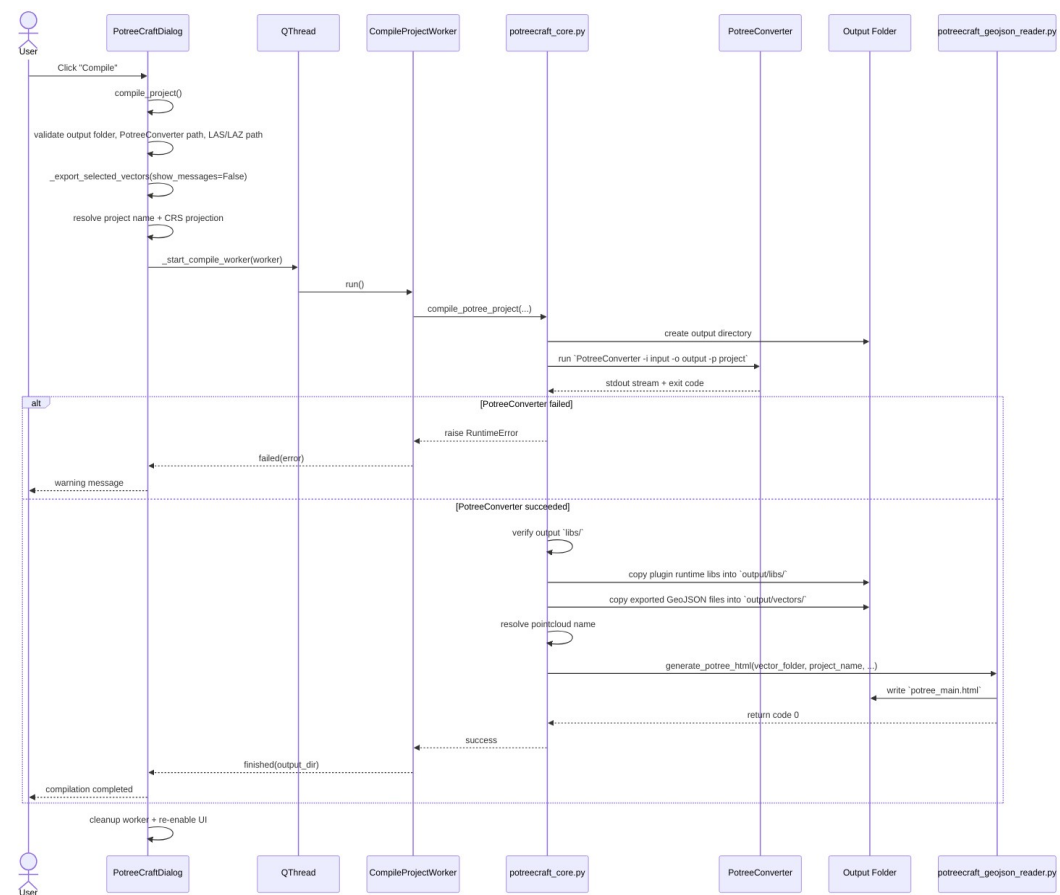


Figure 4.15: Compile project sequence diagram

5. QGIS PLUGIN DEVELOPMENT: TESTING THE TOOL

5.1 Testing the tool: the sample “roadsection” dataset

5.1.1 Pointcloud data

The „roadsection.las” pointcloud is a small sample of a project done with a vehicle-mounted scanner. The area contained by the bounding box covers about 287.4 m x 97.3 m horizontally, with about 55.0 m of vertical range, but that is only true because of a set of residual points appearing on the scan a few hundred meters away, probably due to improper cleanup. The pointcloud in reality contains useful information only about a roughly 95 m long part of the road designated as 6233 main road leading to the M6 highway connection near the south side of Paks (see approximate measurements in Figure 5.2, and Figure 5.3).

Unfortunately the metadata section of the .las file is also missing a few things, but since the location of this pointcloud sample is known (near Paks, Hungary). This information helps, because the only locally used CRS in modern cartography / surveyor work is the “Egységes Országos Vetület” (Unified National Projection) or EOVS for short (EPSG:23700). Knowing this, it is not hard to guess that the metadata we’re missing in the srs section of the pdal metadata report contain that, as seen in Figure 5.1.

```

37  "srs":
38  {
39    "compoundwkt": "",
40    "horizontal": "",
41    "isgeocentric": false,
42    "isgeographic": false,
43    "json":
44    {
45    },
46    "prettycompoundwkt": "",
47    "prettywkt": "",
48    "proj4": "",
49    "units":
50    {
51      "horizontal": "unknown",
52      "vertical": ""
53    },
54    "vertical": "",
55    "wkt": ""
56  },

```

Figure 5.1: srs block of the pdal info --metadata command output for roadsection.las

The .las file contains rgb color data, and intensity values, so coloring the dots through elevation numbers is not our only option.

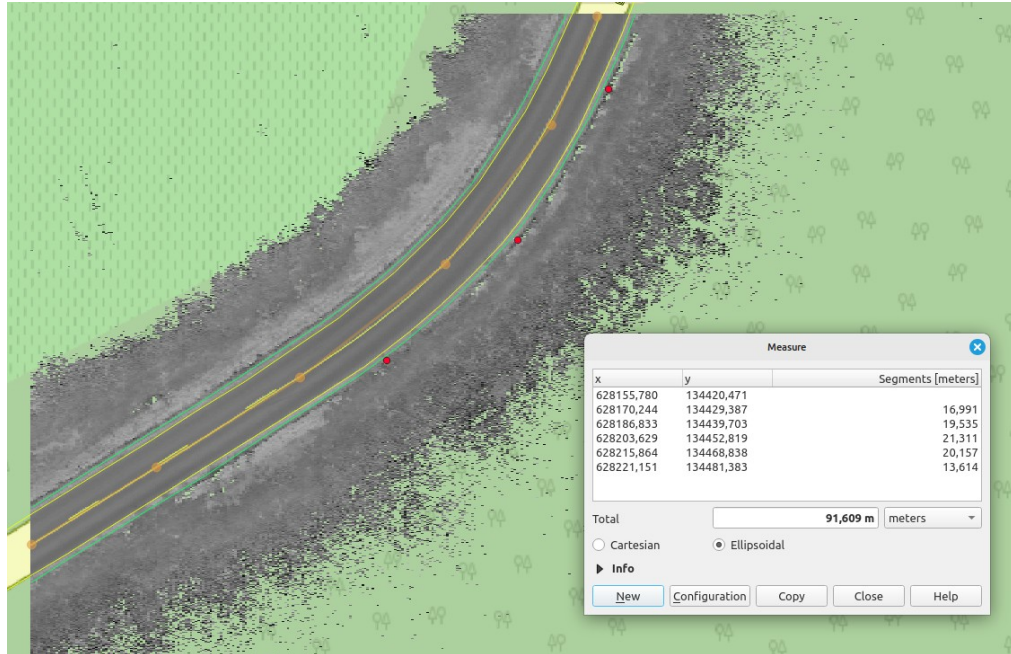


Figure 5.2: Approximate measurement of road length in QGIS

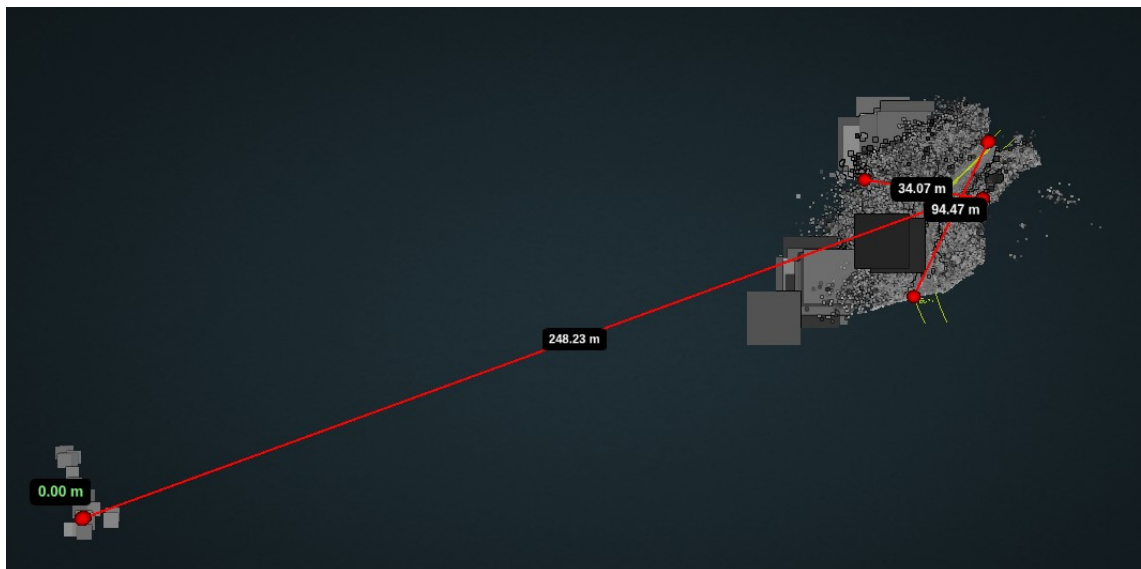


Figure 5.3: Approximate measurement or the distance of residual points from the road surface in potree

5.1.2 Vector data

For this demonstration, I took inspiration from my junior years, and this collection of test vector data was entirely created by hand. While I did not have such purpose-made tools for the job, like the PCS software I used to work with during this phase of the mapping process, potree itself can be a clunky, but effective substitute for this: using the measure tool we're allowed to put down points, which later can be exported in a json format. Those points can be easily transformed into visually impressive 3D vector data with QGIS later on.

The following layers were created for this demonstration:

- `road_edges` (Linestring): A line on both sides of the asphalt, marking the end of the road surface.
- `road_fence` (Linestring): The line follows the top of the guardrails at the edges of the road where present.
- `road_paint` (Linestring): This layer contains the markings on the road.
- `road_signs` (Point): This layer contains the top point of each traffic sign found in this pointcloud (a total of 3).
- `road_number_fat_z` (Polygon): This layer extends to the empty space outside of the bounding box. It was drawn in QGIS, based on a simple linestring layer where I carved the number "6233" onto the road. The polygon nature was achieved by running the "Buffer" algorithm on this linestring layer. After that, a constant Z elevation was added to each breakpoint to elevate the numbers up to an approximate 148 meters above sea level, to be in line with the rest of the layers.

5.1.3 Putting the project together

When designing the plugin I was trying to put together an interface which is relatively intuitive, but at the same time I had to include some of the more technical aspects, as the

power lies within the possible parameter combinations the command line tools were designed to take.

When we open the plugin, it is visible from the start, that the interface is split into settings groups, such as:

- Project vector layers: provides an overview of the current vector layer imported to the project. We can also enable/disable a layer for export, set the marker circle radius for point layer export, and set the annotation titles and descriptions (currently disabled in the build, as it is a feature not fully implemented yet).
- Point cloud and output: This section is where we have to set up the core dependency of the plugin, and set the output project folder where we'd like to redirect the potreeconverter output, and enhance the results. We also have to select a pointcloud file we're going to work with, that reference is going to be used in the following section as well.
- LAS to GeoTiff import: This section serves a similar purpose to the previous one, the difference is that the settings here refer to the QGIS raster import of the pointcloud, and does not influence the settings we set for potreeconverter.
- Actions: The final section has two buttons as of now, first, a conversion button which automatically converts shape layers to geojson format, and the "compile" button which sends every bit of information we set up before to the potreeconverter and the ecosystem the plugin contains around it.

Knowing this, the use of the plugin is fairly simple:

1. Select the location of the potreeconverter on our PC
2. Select the location of the pointcloud we'd like to work with
3. Import a raster preview of the pointcloud for us to see what we're working with
4. Import, generate, draw or edit some vector files we'd like to add to the potree scene.
5. Adjust the rest of the settings according to the project requirements.
6. Compile the output

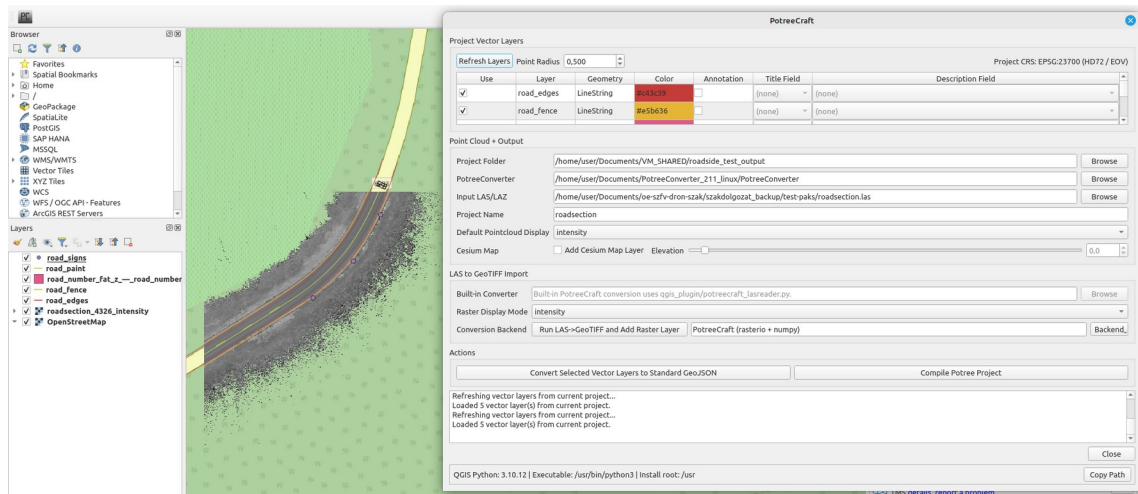


Figure 5.4: The plugin interface while putting together the "roadsection" dataset part of the demonstration

After the project conversion process is done, we need to navigate to the output directory in a terminal and start up a webserver. I recommend either python's webserver:

```
python3 -m http.server 8095
```

or the node js one:

```
http-server -p 8095
```

Since potree itself is a web-based pointcloud viewer this step is mandatory for local testing. When the server is running, we'll be to see our work at the following URL:

http://127.0.0.1:8095/potree_main.html

The address "127.0.0.1" is also known as the localhost address, which basically points at the PC we're using at the moment, so it is often used in local tests and self-hosted applications. The port "8095" can be modified to anything which is not currently in use on the given machine. "potree_main.html" is the enhanced output html document the plugin puts together, where all the vectors, annotations and the possible cesium map layer is located. The output for this example project can be seen in Figure 5.5 and Figure 5.6. In the raw code there's an option to set a thicker line width for our vectors, as it is seen in Figure 5.7, however due to technical limitations out of my control, this feature only works on certain graphics cards and drivers. In my case line thickness appears cor-



Figure 5.5: A screenshot of the "roadsection" dataset in potree, with vectors and pointcloud visible.

rectly in browsers running on linux (and android) systems, however thickness always stays 1 pixel in width on Windows PC-s.

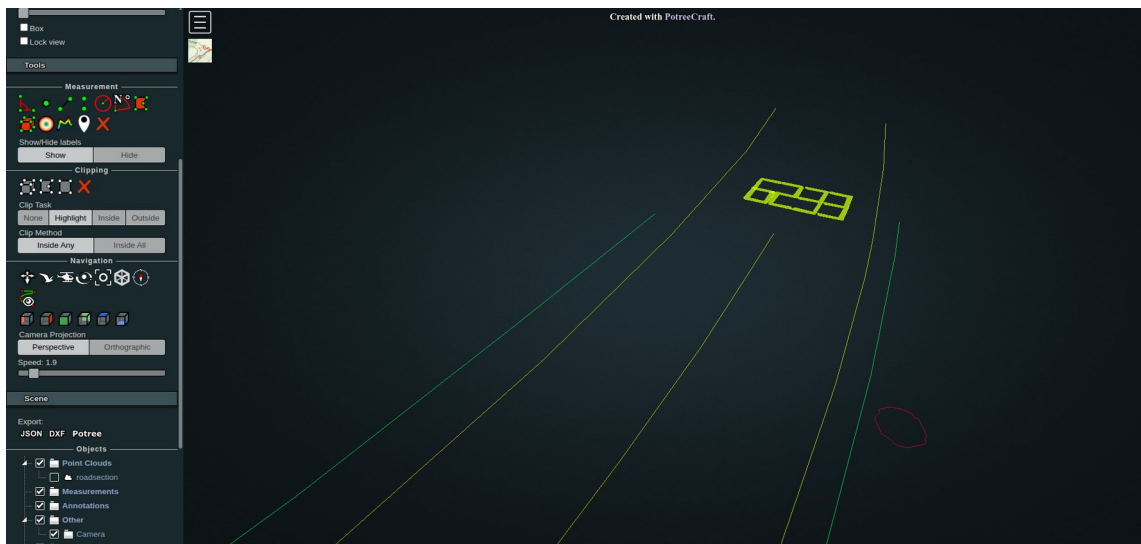


Figure 5.6: A screenshot of the "roadsection" dataset in potree, with vectors visible, but pointcloud turned off.

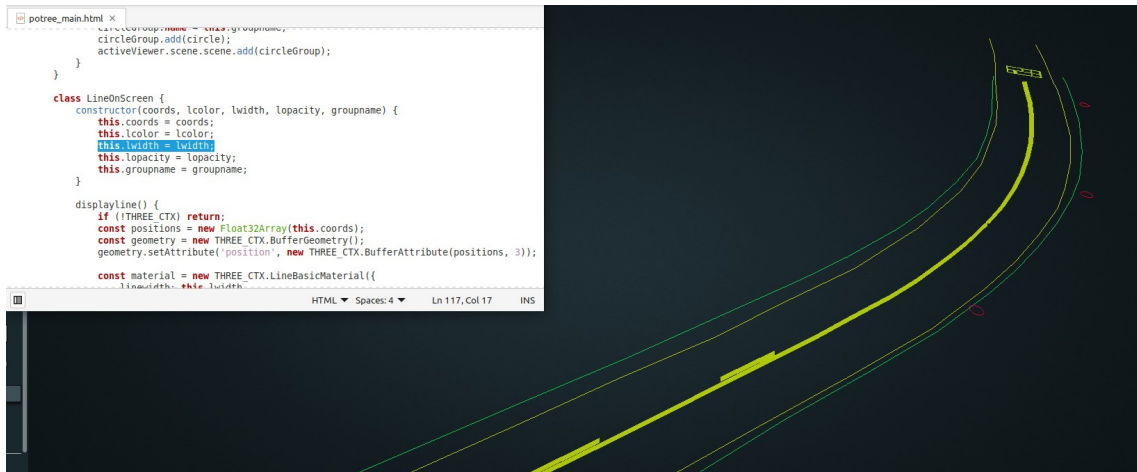


Figure 5.7: Line width parameter for vectors (does not work everywhere).

It is possible to add an OpenStreetMap layer to any project, as seen in Figure 5.8. Users can also select an elevation level for the layer, so the pointclouds can directly “sit” above the selected map layer. This feature relies on the previously mentioned Cesium library, which we copy in as an additional dependency into the compiled projects.

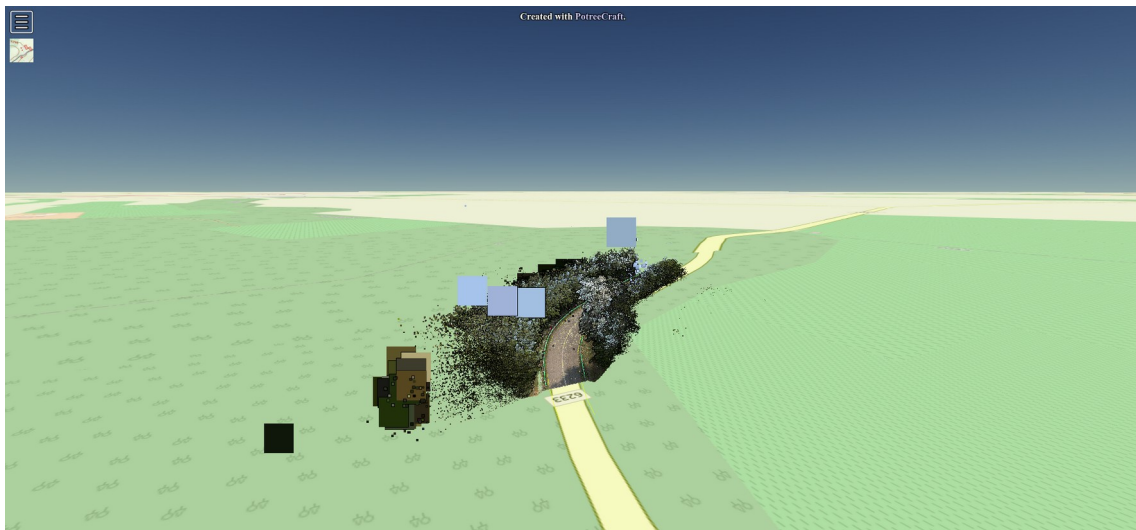


Figure 5.8: Roadsection dataset with Cesium OpenStreetMap layer underneath the pointcloud, elevated to 147 meters ASL.

5.2 Testing the tool: Self-collected dataset on Határi dűlő, Pomáz

This dataset can be hardly called a professional one, however the reason why I decided to include it is to showcase exactly why it is the case, instead of just throwing the data out as digital garbage. Or at least have some scientific reasoning before doing so.

5.2.1 Pointcloud data

The location covered by this dataset is between Pomáz and Szentendre, a flatland called Határi dűlő, which mostly used as a pasture for local ranchers. This area is going to be the topic of discussion for a colleague of mine, Sándor Hingyi, who is focusing on this area as part of his thesis, “UAV felszín borítottsági vizsgálatok” (UAV surface coverage studies)[8].

On the 1st of March, 2026, we went out to look around and to test out a DJI Mini 3 drone I’ve purchased before. The Mini 3 is a C0 category hobby drone weighing just 249 grams, and it comes with the DJI Fly controlling software on android. Compared to the DJI Pilot series – which is reserved for professional drones – this app is heavily neutered, and paired with a DJI Mini 3, it is not capable of having even simple line-based flight paths, let alone supporting anything close to area-based aerial surveying. For this reason the flight path the drone took above this field was an improvised one, with a 3 second timer between taking automatic photographs.

During this short flight, the drone managed to take 28 photos from a height of around 80 to 83m AGL. Originally I’ve used these photos to try out the capabilities of the open source WebODM software package, which interestingly contains a fork of potree itself for displaying 3D models after we ran a successful photogrammetry process. Reports of this conversion can be found in [INSERT APPENDIX REFERENCE].

After this process finished, I was able to download a .las copy of the cloud the software generated from the photos and the metadata (as seen in Figure 5.9). The only major difference between this pointcloud and the roadsection dataset, is that this is missing the intensity values, so only RGB and elevation-based display modes can be used here.

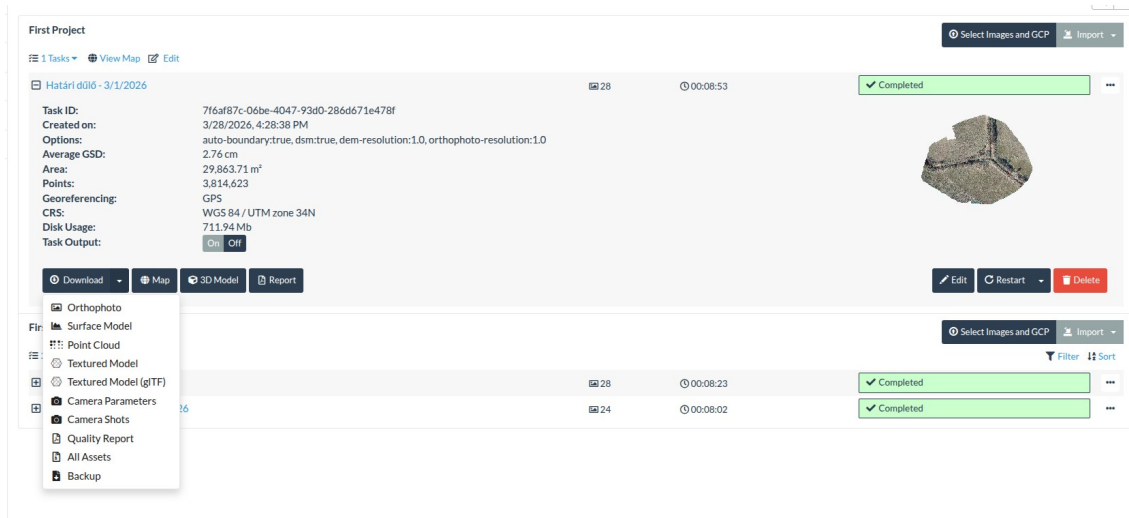


Figure 5.9: WebODM interface, post processing

5.2.2 Vector data

The goal here was to generate two sets of vector data: first, to add some contour lines to illustrate topographic height with vectors as well, and the second to add a series of data-points retracing the flight path, camera locations and the corresponding photograph's location in 3D space.

The following vector layers were used in this demonstration:

- `joined_layer_eov` (Point): This layer represents a point in space where a photograph was taken. It was created from the „camera-shots” geojson file.
- `flight_path_eov` (Linestring): This layer is the reconstructed path of the drone. It was derived from “`joined_layer_eov`”, and tied together into a linestring based on the “`capture_time`” field.
- `camera_shots_linestring_eov_z_drape` (Linestring): This layer contains the approximation of the photographs' location on the map.
- `contour_lines_z_drape` (Linestring): This layer contains the result a set of topographic contour lines.

In the following section I'll discuss how both of these were created in detail using QGIS.

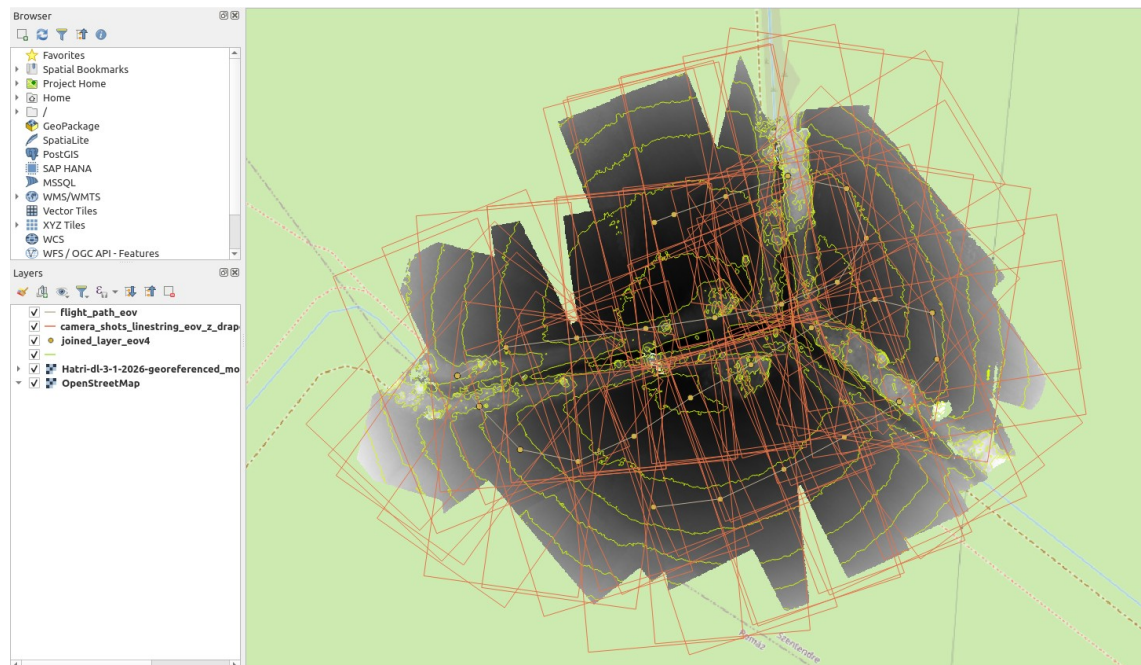


Figure 5.10: Vectors of the Határi dűlő dataset

The initial dataset

As I've mentioned the base set of picture were taken as part of an improvised flight taken with a hobby drone, with no official software support for waypoint flights. On top of that, the area did not contain any ground control points either, which further reduces the resulting model's accuracy. In Figure 5.11 the path taken by the drone can be seen, which immediately should raise some concerns on the overlap values between photographs taken. In Karl Kraus' Photogrammetry, chapter 3.8, the professor advised the readers to calculate routes with $q = 30\%$, and $l = 60\%$ (side and forward overlap) to have sufficient overlap between photographs taken[9]. (The equations used here can be found on page 118. of the 1998. Hungarian translation of the book.) To visualize the problem with our improvised route, I've drafted up the edges of the photographs as rectangles on the map, and use them as the vector layers for this second demonstration.

```

{
  "dji_fc3682_4000_2250_brown_0.6666": {
    "projection_type": "brown",
    "width": 4000,
    "height": 2250,
    "focal_x": 0.6493349021020803,
    "focal_y": 0.6493349021020803,
    "c_x": -0.00364640912002432,
    "c_y": 0.0014308454855519182,
    "k1": 0.05134432106710347,
    "k2": -0.08661699579502441,
    "p1": -0.0005759111377238758,
    "p2": -0.0003061968194534967,
    "k3": 0.03316994433795165
  }
}

```

Listing 5.1: Contents of Hatr-dl-3-1-2026-camera-params.json

As a secondary vector dataset I've also attempted to draw up a set of contour lines for every 1 meter difference in elevation.

Making something out of nothing

From the camera-params.json, seen in Listing 5.1 we can get the basic parameters of the camera used on the drone. Focal value is around 0.6493, image width is 4000 pixels, height is 2250 pixels. We can also easily check that the drone flew around 81-83 meters above ground level. To get the camera constant value (c), we need to know the width of the sensor on the drone, which is around 14.08×10.56 mm-s on the Mini 3.

$$c = 0.6493 * 14.08 = 9.142144 \text{ mm} = 0.009142144 \text{ m}$$

Calculating scale:

$$m_h = h / c = 82 / 0.009142144 = 8969,44961707 \rightarrow \sim 8970$$

Getting the approximate footprint of rectangles:

$$S_x = s_x * m_h = 0,01408 * 8970 = 126,2976 \rightarrow \sim 126 \text{ m}$$

$$S_y = s_y * m_h = 0,01056 * 8970 = 94,7232 \rightarrow \sim 95 \text{ m}$$

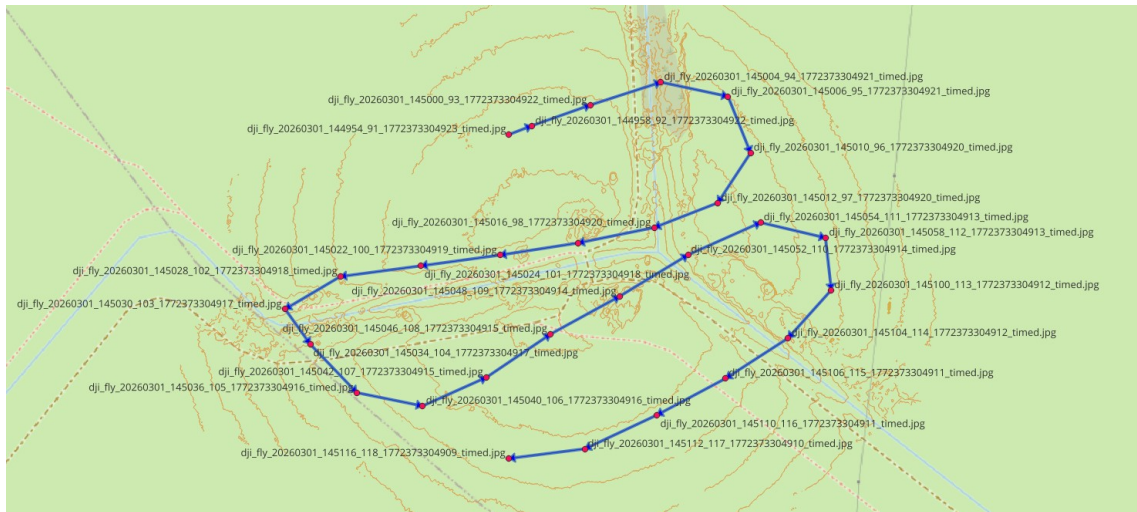


Figure 5.11: Location of pictures taken, shown as a path

After knowing these numbers, we can start working on the geometries.

First we need to import our initial point layer from the file „Hatri-dl-3-1-2026-camera-shots.geojson“. With the „Points to path“ tool in the processing toolbox we created the linestring layer showing our path. We need to explode this linestring layer to individual segments with the „Explode lines“ tool.

On the exploded lines we need to create a new field, which we can name “heading”, by using the following command in field calculator:

```
degrees( azimuth( start_point($geometry), end_point($geometry) ) )
```

This way we have a field which contains information about the rotation of the photographs’ rectangles in degrees.

Using the “Join attributes by nearest” tool we can then bring this value back to our point layer (creating a “joined layer” temporary layer, which I just decided to continue to use to distinguish my “enhanced” point layer from the base one). After some styling and labeling, the point and linestring layer can be seen in Figure 5.11.

Following this, we can just simply draw up the approximate places of our photographs using the “Geometries by expression” tool.

```

with_variable(
  'w', 126,
  with_variable(
    'h', 95,
    rotate(
      make_polygon(
        make_line(
          array(
            make_point(x($geometry) - @w/2, y($geometry) - @h/2),
            make_point(x($geometry) + @w/2, y($geometry) - @h/2),
            make_point(x($geometry) + @w/2, y($geometry) + @h/2),
            make_point(x($geometry) - @w/2, y($geometry) + @h/2),
            make_point(x($geometry) - @w/2, y($geometry) - @h/2)
          )
        )
      ),
      "heading",
      $geometry
    )
  )
)

```

Listing 5.2: The script drawing rectangles, which we need to input in the "Geometries by expression" tool

To add some secondary vector dataset, I've also imported the file "Hatri-dl-nadr-3-1-2026-orthophoto.tif", which was created by WebODM as a raster underneath the tif elevation layer created with the PotreeCraft plugin. We then find the contour line creation menu in Raster → Extraction → Contour, which will create 2D linestring layers based on the elevation tif. To give back some sort of elevation value to these lines, we can select the "Drape (set Z value from raster)" algorithm from the processing toolbox to plant these lines onto the pointcloud. Unfortunately this method is not perfect, since there will be some anomalies, some breakpoints having 0 elevation at the edges for example, but I think it is good enough for a quick demonstration. Our QGIS project now should look something like in Figure 5.12.

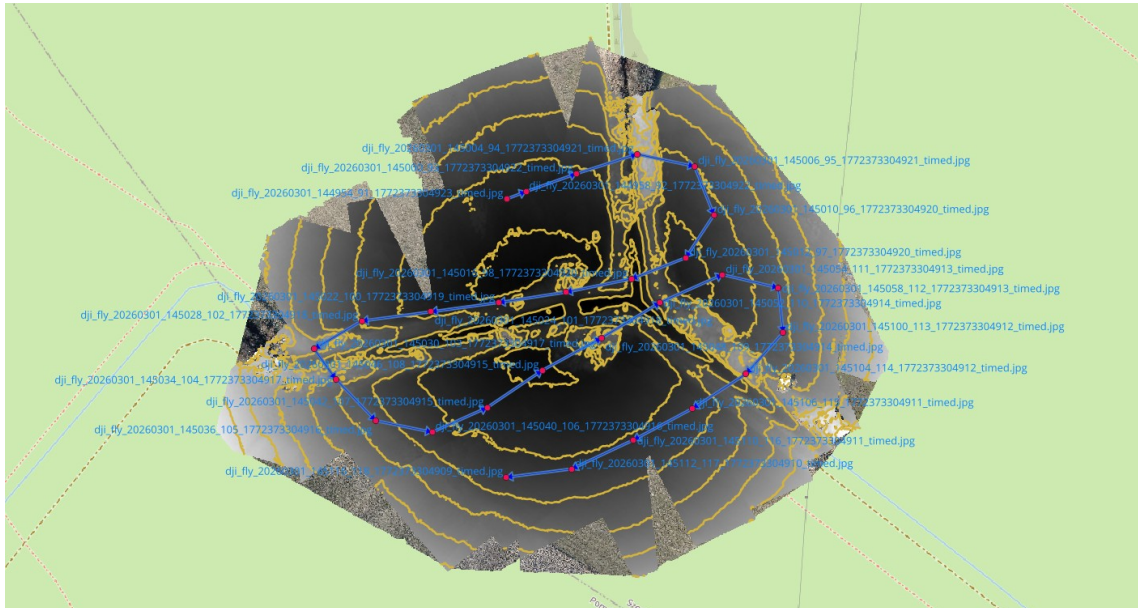


Figure 5.12: Contour lines made from elevation raster layer

If we turn back the display of the rectangles, the result should be as seen in Figure 5.13.

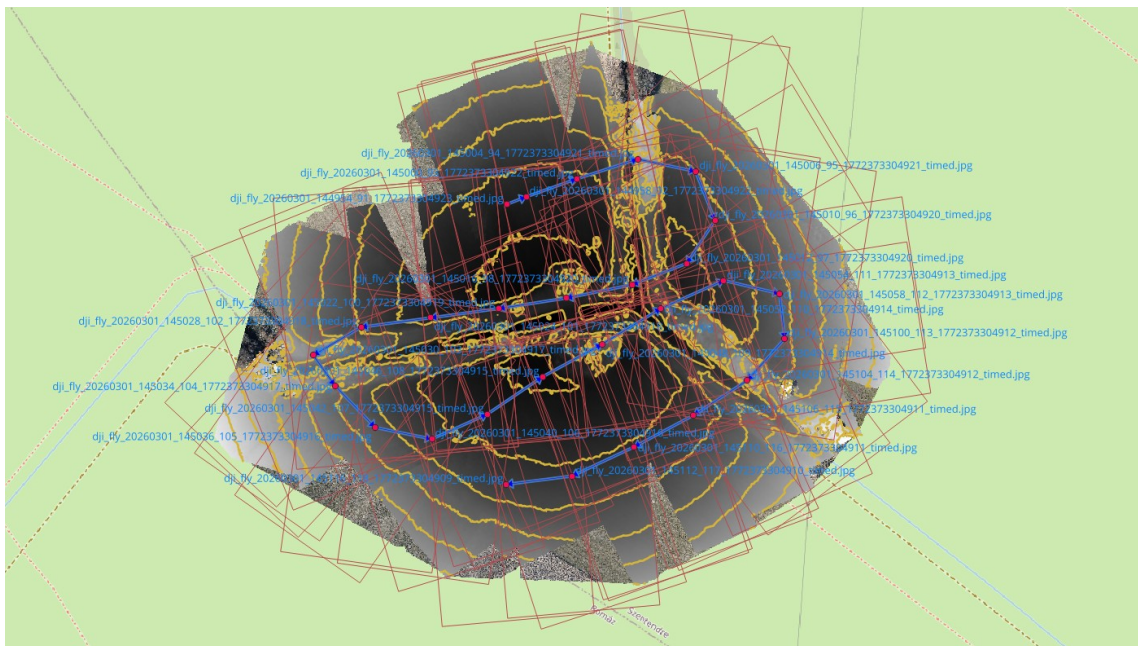


Figure 5.13: Contour lines with approximate photo location rectangles above raster layers.

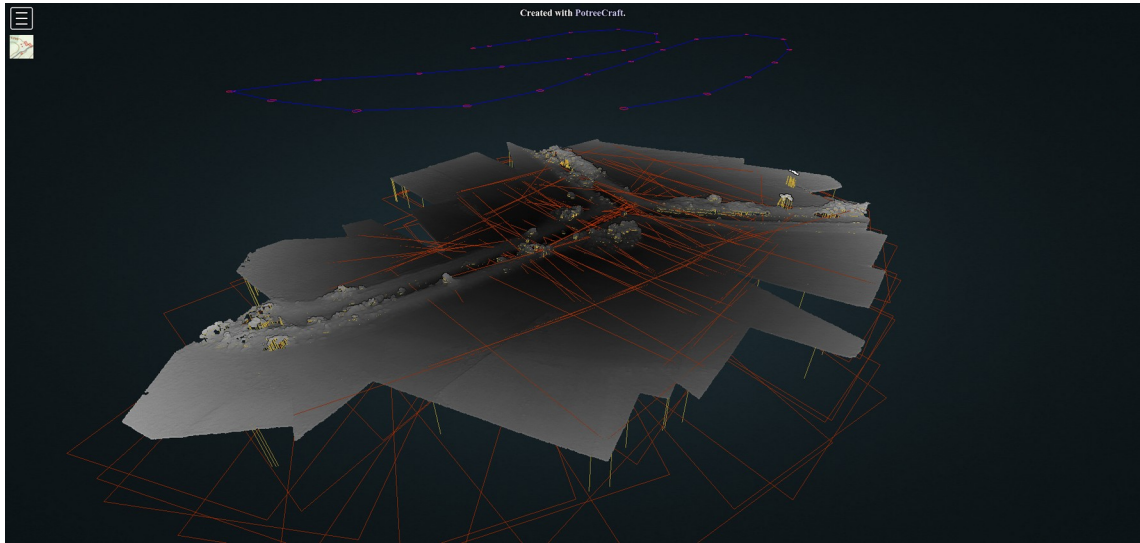


Figure 5.14: Potree view of the area, with grayscale elevation-based coloring

After conversion to Potree, with the discussed methods in chapter 5.1.3, the test dataset would look like as it is shown in Figure 5.14, Figure 5.15, and Figure 5.16. A closer shot of the draped contour lines can be seen in Figure 5.17, showing the imperfect nature of the conversion, as many lines finish in dropping down to 0 elevation.

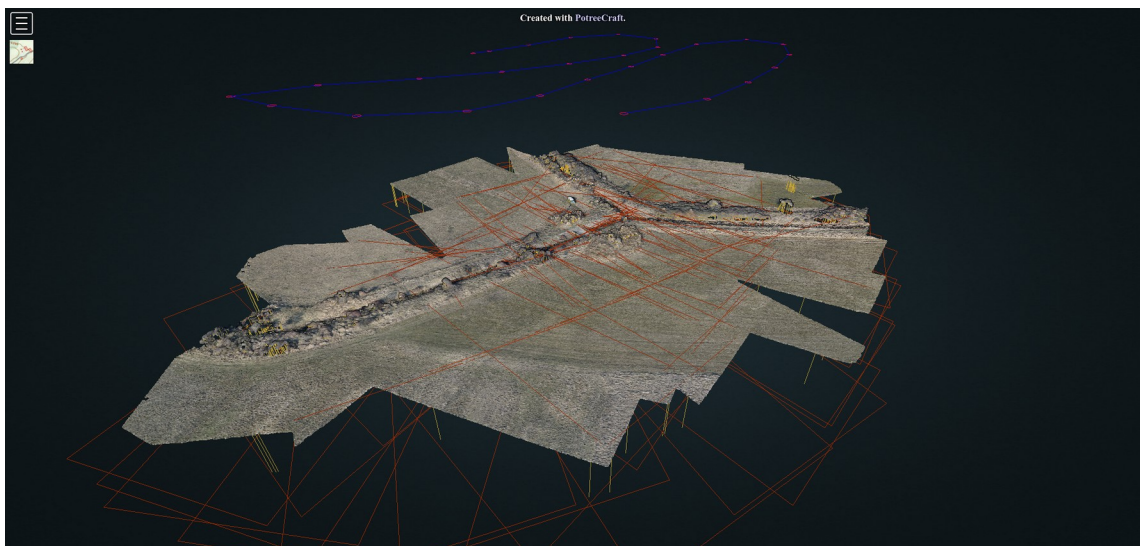


Figure 5.15: Potree view of the area, with RGB-based coloring

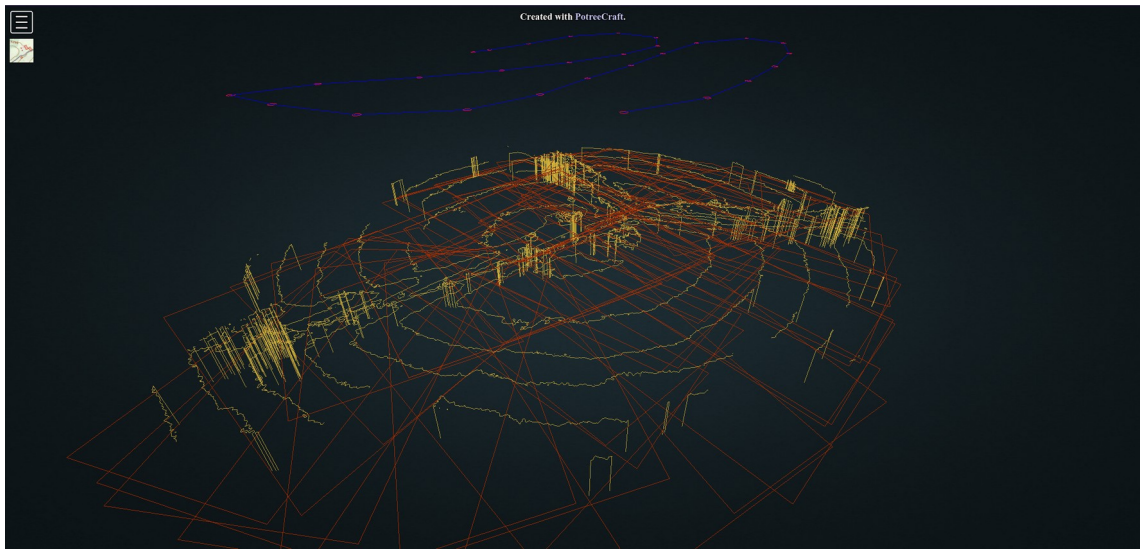


Figure 5.16: Potree view of the area, vectors only

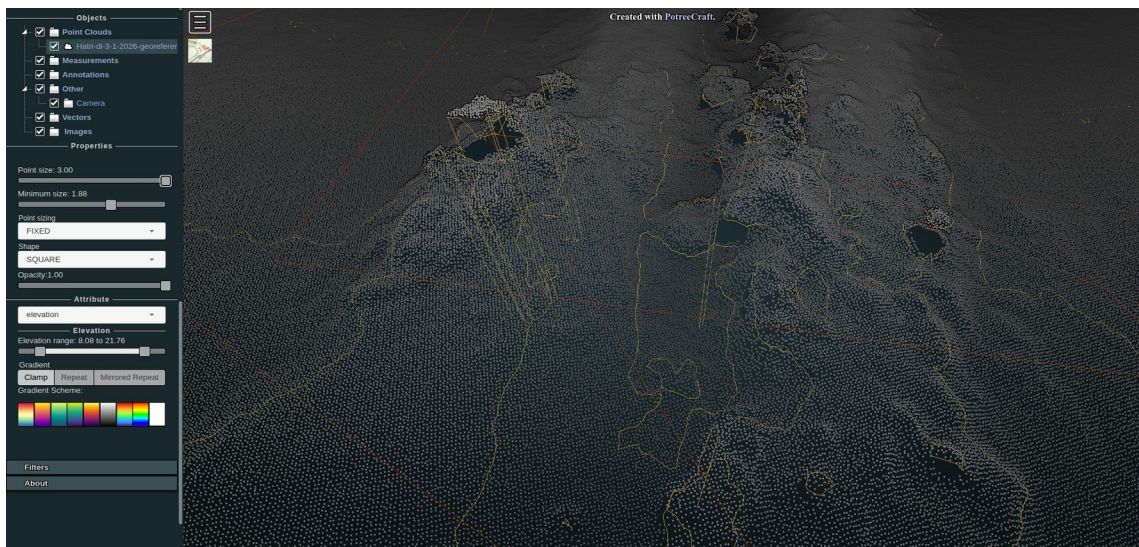


Figure 5.17: Potree view of the area, a closer view of contour line, with decreased point size, to increase vector visibility

Overlap analysis

After examining the overlap of each rectangle, and classifying them using the following simple classification method seen in Listing 5.3, the results are quite underwhelming.

```

if l_front >= 80.0 and q_side >= 70.0:
    risk = "good"
elif l_front >= 75.0 and q_side >= 60.0:
    risk = "usable"
elif l_front >= 65.0 and q_side >= 50.0:
    risk = "weak"
else:
    risk = "terrible"

```

Listing 5.3: Simple overlap percentage classification

There were only 3 rectangles which managed to get into the „weak” category, the rest are all below 65% front and / or less than 50% side overlap (Listing 5.4). The report sheet can be found in the appendix at A.II.IV: overlap_report.csv.

Summary:

Features: 28

Mean front_best: 60.67

Median front_best: 59.17

Min front_best: 54.20

Mean side_best: 60.08

Median side_best: 59.50

Min side_best: 44.78

Risk classes: {'terrible': 25, 'weak': 3}

Listing 5.4: Summary of overlap statistics

At this point we can conclude that manual flying has provided an inconsistent and unreliable dataset, which not just resulted in a pointcloud shaped like a metal shrapnel, it probably contains distortions as well, which we cannot confirm due to the lack of ground control points or other scans done in the area. But at least the flight served well for me to learn the basics of the WebODM software package, and also it gave me a great base material for the demonstration of this plugin.

6. CONCLUSION

6.1 The direction of further development

The current version of PotreeCraft is available on the official QGIS plugin repository, which means it can be downloaded through QGIS itself, by selecting it from the plugin installer menu. As of writing this, currently the initial release of version 2.x.x is at 69 downloaded copies of the experimental version of 2.1.2.

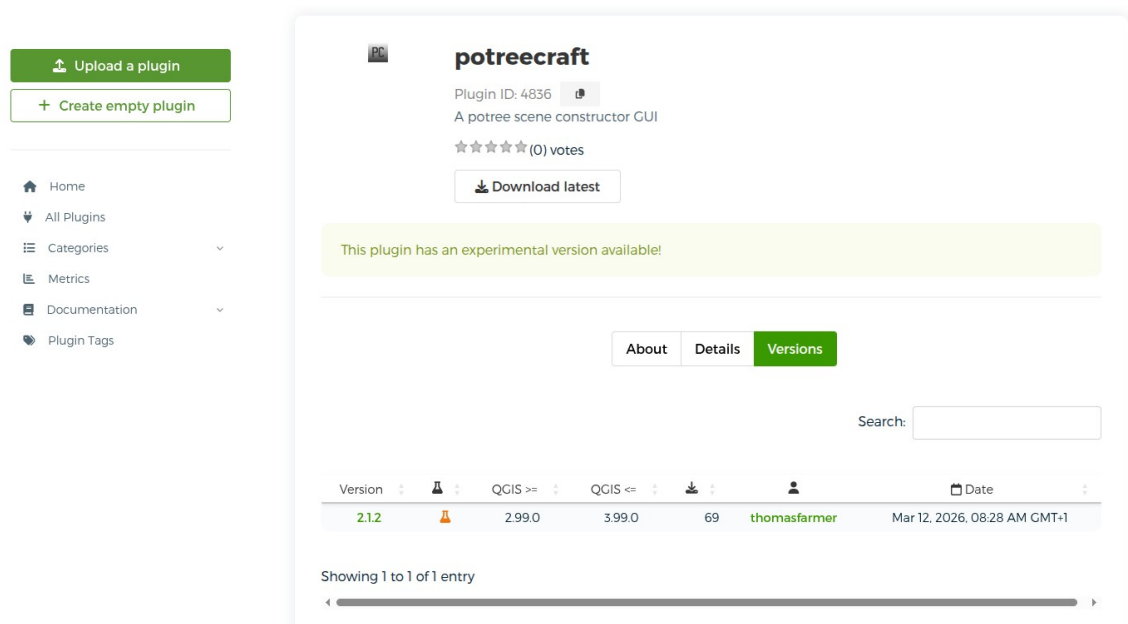


Figure 6.1: PotreeCraft on the official QGIS plugin repository

6.1.1 What is still missing

Version 1.x.x series supported a variety of features which is not currently implemented, or just partially implemented in the 2.1.2 version. Just to mention a few:

- The old version allowed the user to integrate vectors as parts of the scene, so they'd show up in the list, and users can turn off each vector layer one by one. This is currently not the case with 2.1.2.

- Point layers used to be represented as mesh spheres or disks for increased visibility. This might affect performance with large datasets, so currently I've decided to draw circles instead.
- Similarly, point layers currently have no option to be imported as annotation layers, but that is one of the key features which was present in the old versions.
- The blast2dem process wrapper had way more customizability, taking advantage of the wide set of parameters the binary can take as input.

6.1.2 Known bugs

- Version 2.1.2 does not process correctly the checkboxes within the plugin assigned to vector layers. This means that we must remove a vector layer from the project itself if we'd like to exclude that layer from the project export. (Fixed in 2.1.3.)
- On Windows when using blast2dem as the raster generating backend, the user gets zero information about the status of the process, only a black screen appears with no logs in it. The user has to wait an unknown amount of time, depending on the size of the dataset and the processing capabilities of the computer.

6.1.3 Direction of development

With all honesty, there's quite a few ways I can think of which would improve the project in one or another way, from small UI improvements to more flushed out features. Here's a short list of the ones I found important or interesting:

- The UI is getting large and crowded, you generally cannot see your workspace and the plugin interface at the same time, unless you work on multiple screens. I probably have to return to the older, tab-based approach to conserve space and make the interface less cluttered.
- Potree by default offers the user to put down many of the tools listed on the palette, measure points, angles, etc. There could be a set of settings automatically preparing these measurements in the scene after compilation.

- One thing I haven't really had the chance to implement even in the 1.x.x versions is support for labels, or any sort of vectorized text appearing in the scene, not just in form of annotations.

7. SUMMARY

7.1 Summary

PotreeCraft is a project that aims to solve a real problem by serving as a bridge between land surveyors, cartographers, and GIS developers working with Potree. This plugin provides a GUI wrapper around the well-known point cloud viewer tool, enabling vector data integration without requiring any knowledge of JavaScript.

When writing this, my aim was to provide a detailed overview not only of the inner workings of the plugin and how to use the tool I created, but also to summarize the necessary knowledge required to set up the development environment, in case someone would like to try plugin development and contribute to the QGIS community.

Finally, I hope that the contents of Chapter 5 have convinced the reader that the project indeed has real-world applications and that it is a worthwhile goal to keep the small user base alive by maintaining and expanding the project over time.

7.2 Összefoglaló (Summary, Hungarian)

A PotreeCraft egy olyan projekt, amely egy valós problémát igyekszik megoldani azáltal, hogy hidat képez a Potree-val dolgozó földmérők, térképészek és GIS-fejlesztők között. Ez a bővítmény egy grafikus felhasználói felületet biztosít a jól ismert pontfelhő-megjelenítő eszköz köré, lehetővé téve a vektoros adatok integrációját JavaScript-ismeret nélkül.

A dolgozat írásakor az volt a célom, hogy ne csak a bővítmény működésének részletes bemutatását és az általam készített eszköz használatát ismertessem, hanem összefoglaljam a fejlesztői környezet létrehozásához szükséges ismereteket is, abban az esetben, ha valaki szeretné kipróbálni magát a bővítményfejlesztésben, és hozzájárulni a QGIS közösséghez.

Végül remélem, hogy az 5. fejezet tartalma meggyőzte az olvasót arról, hogy a projektnek valóban vannak gyakorlati alkalmazásai, és hogy érdemes cél a viszonylag kis felhasználói bázis fenntartása, valamint a projekt folyamatos fejlesztése és bővítése az idő múlásával.

8. BIBLIOGRAPHY

Bibliography

- 1: Markus Schütz, Potree: Rendering Large Point Clouds in Web Browsers, 2016
- 2: QGIS project, PyQGIS Developer Cookbook, 2002
- 3: Béres Tamás, PotreeCraft Github repository, 2019, <https://github.com/ThomasFarmer/PotreeCraft>
- 4: GIS Stack Exchange, How to import qgis.core to Python ap, 2012, <https://gis.stackexchange.com/questions/22886/how-to-import-qgis-core-to-python-app>
- 5: Gary Sherman - Jonah Sullivan, pb_tool github page, 2014, https://github.com/g-sherman/plugin_build_tool
- 6: RedHat, What is CI/CD?, 2025, <https://www.redhat.com/en/topics/devops/what-is-ci-cd>
- 7: Guido van Rossum et al., PEP 8 – Style Guide for Python Code, 2001, <https://peps.python.org/pep-0008/>
- 8: Sándor Hingyi, UAV felszín borítottsági vizsgálatok, 2026
- 9: Karl Kraus, Photogrammetry, 1998

9. TABLE OF REFERENCES

Table of Figures

Figure 3.1: Python interpreter crashes on launch after OSGeo4W installation....	7
Figure 3.2: Testing if IntelliSense is capable of reading the qgis package.....	9
Figure 3.3: Installation command for pb_tool.....	10
Figure 3.4: Qt5 designer.....	10
Figure 3.5: QGIS installation commands on a linux system.....	11
Figure 3.6: devtools and Qt package installation commands on a linux system...	12
Figure 3.7: pdal installation commands on a linux system.....	12
Figure 3.8: pb_tool installation commands.....	13
Figure 3.9: Plugin reloader.....	14
Figure 4.1: The development loop.....	16
Figure 4.2: installation command and basic usage of ruff.....	19
Figure 4.3: installation command for testing tools.....	20
Figure 4.4: plugin compilation command.....	21
Figure 4.5: plugin deployment command.....	22
Figure 4.6: command for packaging the plugin with pb_tool.....	23
Figure 4.7: High level overview of the plugin.....	25
Figure 4.8: Class diagram for potreecraft.py.....	27
Figure 4.9: Class diagram for potreecraft_dialog.py.....	28
Figure 4.10: Class diagram for potreecraft_core.py.....	29

Figure 4.11: Class diagram for potreecraft_lasreader.py.....	30
Figure 4.12: Log file snippet of the python http.server module.....	32
Figure 4.13: Raster import sequence diagram.....	33
Figure 4.14: Vector export sequence diagram.....	34
Figure 4.15: Compile project sequence diagram.....	35
Figure 5.1: srs block of the pdal info --metadata command output for roadsection.las	36
Figure 5.2: Approximate measurement of road length in QGIS.....	37
Figure 5.3: Approximate measurement or the distance of residual points from the road surface in potree.....	37
Figure 5.4: The plugin interface while putting together the "roadsection" dataset part of the demonstration.....	40
Figure 5.5: A screenshot of the "roadsection" dataset in potree, with vectors and pointcloud visible.....	41
Figure 5.6: A screenshot of the "roadsection" dataset in potree, with vectors visible, but pointcloud turned off.....	41
Figure 5.7: Line width parameter for vectors (does not work everywhere).....	42
Figure 5.8: Roadsection dataset with Cesium OpenStreetMap layer underneath the pointcloud, elevated to 147 meters ASL.....	42
Figure 5.9: WebODM interface, post processing.....	44
Figure 5.10: Vectors of the Határi dúlő dataset.....	45
Figure 5.11: Location of pictures taken, shown as a path.....	47
Figure 5.12: Contour lines made from elevation raster layer.....	49
Figure 5.13: Contour lines with approximate photo location rectangles above raster layers.....	49

Figure 5.14: Potree view of the area, with grayscale elevation-based coloring....	50
Figure 5.15: Potree view of the area, with RGB-based coloring.....	50
Figure 5.16: Potree view of the area, vectors only.....	51
Figure 5.17: Potree view of the area, a closer view of contour line, with decreased point size, to increase vector visibility.....	51
Figure 6.1: PotreeCraft on the official QGIS plugin repository.....	53

Table of Listings

Listing 1.1: The main script section of the potree boilerplate code.....	3
Listing 3.1: Example batch file for OSGeo4W Python interpreter launch.....	8
Listing 3.2: Batch file for launching VS Code using the OSGeo4W Python interpreter	8
Listing 4.1: pb_tool create output.....	17
Listing 4.2: ruff linting error example.....	20
Listing 4.3: the Makefile of the PotreeCraft project.....	24
Listing 5.1: Contents of Hatr-dl-3-1-2026-camera-params.json.....	46
Listing 5.2: The script drawing rectangles, which we need to input in the "Geometries by expression" tool.....	48
Listing 5.3: Simple overlap percentage classification.....	52
Listing 5.4: Summary of overlap statistics.....	52

A. APPENDIX

A.I Class method descriptions

A.I.I potreecraft.py

__init__(self, iface)

This initializes the plugin instance and store the QGIS interface reference, loads translation resources when available, and creates the plugin toolbar and stores basic plugin state.

Inputs:

- *iface*: The active QGIS interface object used to add menus, toolbars, and dialogs.

Output:

- None.

tr(self, message)

Translates a UI string through the Qt translation system.

Inputs:

- *message*: The source string to translate.

Output:

- The translated string returned by ``QCoreApplication.translate(...)``.

add_action(self, icon_path, text, callback, enabled_flag=True, add_to_menu=True, add_to_toolbar=True, status_tip=None, whats_this=None, parent=None)

Creates a QAction and optionally adds it to the plugin toolbar and QGIS plugin menu.

Inputs:

- *icon_path*: Path or Qt resource identifier for the action icon.
- *text*: Visible label for the action.

- `callback`: Function called when the action is triggered.
- `enabled_flag`: Whether the action starts enabled.
- `add_to_menu`: Whether to add the action to the plugin menu.
- `add_to_toolbar`: Whether to add the action to the plugin toolbar.
- `status_tip`: Optional tooltip/status-bar text.
- `whats_this`: Optional help text for Qt's "What's This" system.
- `parent`: Optional parent widget for the action.

Output:

- A configured `QAction`` instance.

initGui(self)

Registers the plugin's main action in the QGIS GUI. It uses the ``add_action(...)`` to wire the main toolbar/menu command to ``run()``.

Inputs:

- None.

Output:

- None.

unload(self)

This removes the plugin menu items and toolbar icons from QGIS during unload.

Inputs:

- None.

Output:

- None.

run(self)

Opens the main PotreeCraft dialog. Uses lazy loading to create the dialog the first time it is opened. It only refreshes vector layers when the dialog already exists.

Inputs:

- None

Output:

- None

A.I.II potreecraft_dialog.py

RasterConversionWorker Function Descriptions

RasterConversionWorker runs LAS/LAZ to GeoTIFF conversion in a background thread so the QGIS dialog stays responsive.

Signals

finished

Emits a result dictionary after a successful conversion.

Output:

- dict: containing conversion details such as backend, output path, return code, and captured command output where applicable.

failed

Emits an error message if conversion fails.

Output:

- str: containing the failure message.

Methods

`__init__(self, backend, input_las, mode, script_path_raw, output_dir=None, output_epsg=None)`

Stores all conversion settings needed by the worker thread.

Inputs:

- backend: Raster backend key, for example *potreecraft* or *blast2dem*.
- input_las: Path to the source .las or .laz file.

- mode: Raster display/value mode such as `rgb`, `intensity`, or `elevation`.
- script_path_raw: Optional raw path string to an external backend executable.
- output_dir: Optional project output directory.
- output_epsg: Optional EPSG code used by the built-in converter.

Output:

- None.

run(self)

Executes the selected backend and emits either *finished* or *failed*.

Inputs:

- None.

Output:

- None directly. (Starts either `_run_blast2dem()` or `_run_potreecraft()` functions.)

_run_blast2dem(self)

Runs the external blast2dem executable and package the result.

Inputs:

- None directly. (Uses worker state set in `__init__`.)

Output:

- dict with:
 - backend: Backend identifier.
 - command: Command list passed to ``subprocess.run``.
 - stdout: Captured standard output.
 - stderr: Captured standard error.
 - returncode: Process exit code.

- `tif_out``: Output GeoTIFF path as a string.

`_run_potreecraft(self)`

Runs PotreeCraft's built-in LAS raster conversion logic.

Inputs:

- None directly. Uses worker state set in `__init__`.

Output:

- dict with:
 - `backend`: Backend identifier.
 - `command`: None, because no external command is executed.
 - `stdout`: Empty string.
 - `stderr`: Empty string.
 - `returncode`: 0 on success.
 - `tif_out`: Output GeoTIFF path as a string.

PotreeCraftDialog Function Descriptions

PotreeCraftDialog is the main QGIS dialog for the plugin. It manages user interaction for vector export, raster conversion, Potree project compilation, logging, and UI state updates.

Core Methods

`__init__(self, iface, parent=None)`

Builds the dialog UI, connects signals, initializes controls, and loads the current project state.

Inputs:

- `iface`: Active QGIS interface instance.
- `parent`: Optional parent Qt widget.

Output:

- None.

log(self, message)

Appends a message to the dialog log and QGIS message log.

Inputs:

- message: Text to record.

Output:

- None.

_log_qgis_message(self, message, level=Qgis.Info)

Sends a message to the QGIS message log under the `PotreeCraft` tag.

Inputs:

- message: Text to record.
- level: QGIS log severity level.

Output:

- None.

_show_warning(self, message)

Logs a warning and show it in a message box.

Inputs:

- message: Warning text.

Output:

- None.

Progress Dialog Methods

_show_conversion_progress_dialog(self)

Creates and show the modal progress dialog for raster conversion.

Inputs:

- None.

Output:

- A QProgressDialog instance.

_show_compile_progress_dialog(self)

Creates and shows the modal progress dialog for project compilation.

Inputs:

- None.

Output:

- A QProgressDialog instance.

Cesium UI Methods

_on_cesium_map_toggled(self, checked)

Enables or disables Cesium-related controls and log the new state.

Inputs:

- checked: Boolean checkbox state.

Output:

- None.

_on_cesium_slider_changed(self, value)

Synchronizes the Cesium elevation spin box from the slider.

Inputs:

- value: Integer slider value.

Output:

- None.

_on_cesium_spinbox_changed(self, value)

Synchronizes the Cesium elevation slider from the spin box.

Inputs:

- value: Floating-point elevation value.

Output:

- None.

Worker Lifecycle Methods

_set_raster_conversion_ui_busy(self, busy)

Enables or disables raster-conversion controls while work is running.

Inputs:

- busy: Boolean for the busy state.

Output:

- None.

_cleanup_raster_worker(self)

Closes progress UI, stops the worker thread, and clears raster-worker references.

Inputs:

- None.

Output:

- None.

_set_compile_ui_busy(self, busy)

Enables or disables compile-related controls while compilation is running.

Inputs:

- busy: Boolean for the busy state.

Output:

- None.

_cleanup_compile_worker(self)

Closes progress UI, stops the compile thread, and clears worker references.

Inputs:

- None.

Output:

- None.

_start_compile_worker(self, worker)

Moves a compile worker into a QThread, connects the signals, and starts the QThread.

Inputs:

- worker: A configured CompileProjectWorker instance.

Output:

- None.

_start_raster_worker(self, worker)

Moves a raster worker into a QThread, connect the signals, and starts the Qthread.

Inputs:

- worker: A configured `RasterConversionWorker` instance.

Output:

- None.

_on_raster_conversion_finished(self, result)

Handles the successful raster conversion, loads the output raster into QGIS, and notifies the user.

Inputs:

- result: Result dictionary emitted by the raster worker.

Output:

- None.

_on_raster_conversion_failed(self, error_message)

Handles the raster conversion errors and restores the dialog state as needed.

Inputs:

- error_message: Readable failure message.

Output:

- None.

_on_compile_finished(self, result)

Handles successful project compilation and informs the user.

Inputs:

- result: Result dictionary emitted by the compile worker.

Output:

- None.

_on_compile_failed(self, error_message)

Handles compilation failure and restores the dialog state.

Inputs:

- error_message: Readable failure message.

Output:

- None.

Path and Status Methods

_update_python_status(self)

Shows the active QGIS Python version and executable path.

Inputs:

- None.

Output:

- None.

_copy_python_path(self)

Copies the active QGIS Python executable path to the clipboard.

Inputs:

- None.

Output:

- None.

_browse_las_input(self)

Lets the user pick a LAS/LAZ input file and fills the project name if empty.

Inputs:

- None.

Output:

- None.

_browse_output_folder(self)

Lets the user pick the project output folder.

Inputs:

- None.

Output:

- None.

_load_potreeconverter_path(self)

Loads the saved PotreeConverter path from QGIS settings into the dialog.

Inputs:

- None.

Output:

- None.

_save_potreeconverter_path(self, path)

Saves the PotreeConverter executable path into QGIS settings.

Inputs:

- path: Path string to persist.

Output:

- None.

_persist_potreeconverter_path(self)

Saves the currently typed PotreeConverter path after editing finishes.

Inputs:

- None.

Output:

- None.

_browse_potreeconverter_path(self)

Lets the user pick the PotreeConverter executable path.

Inputs:

- None.

Output:

- None.

_browse_raster_script(self)

Let the user pick an optional external raster backend executable.

Inputs:

- None.

Output:

- None.

_init_raster_backend_menu(self)

Builds the raster backend selection menu and connects its actions.

Inputs:

- None.

Output:

- None.

_on_raster_backend_action_triggered(self, action)

Switches the active raster backend after a menu selection.

Inputs:

- action: Triggered Qt action carrying the backend key.

Output:

- None.

_update_raster_backend_ui(self)

Refresh labels, placeholders, and enabled states based on the selected raster backend.

Inputs:

- None.

Output:

- None.

Vector Layer Table Methods

refresh_vector_layers(self)

Reloads vector layers from the current QGIS project into the export table.

Inputs:

- None.

Output:

- None.

_update_project_crs_label(self)

Shows the current project CRS in a readable label.

Inputs:

- None.

Output:

- None.

_add_vector_layer_row(self, layer)

Adds one layer row to the table with export, color, and annotation controls.

Inputs:

- layer: QGIS vector layer instance.

Output:

- None.

_layer_from_row(self, row)

Resolves the QGIS layer represented by a table row.

Inputs:

- row: Table row index.

Output:

- The matching layer object, or `None` if the row cannot be resolved.

_geometry_label(layer)

Converts a QGIS geometry type into a display-friendly label.

Inputs:

- layer: QGIS vector layer instance.

Output:

- str: a basic geometry type, such as Point, LineString, Polygon, or Unknown as a fallback state.

_layer_color_hex(layer)

Extracts the layer's renderer color as a hex string.

Inputs:

- layer: QGIS vector layer instance.

Output:

- str: hex color, defaulting to #808080 if unavailable.

_selected_vector_layers(self)

Collects all layers currently checked for export.

Inputs:

- None.

Output:

- A list of (row, layer) tuples.

_sanitized_layer_filename(layer_name)

Converts a layer name into a filesystem-friendly GeoJSON base name.

Inputs:

- layer_name: the original layer name string.

Output:

- str: similar to original layer name, but with spaces replaced by underscores.

_clear_previous_vector_exports(self, vector_out_dir, selected_layers)

Removes stale GeoJSON exports that no longer correspond to selected layers.

Inputs:

- vector_out_dir: Export directory containing .geojson files.
- selected_layers: List of currently selected layer tuples.

Output:

- None.

_embed_layer_style_metadata(self, geojson_path, color_hex)

Writes the PotreeCraft color metadata into an exported GeoJSON file.

Inputs:

- geojson_path: Path to the exported GeoJSON file.
- color_hex: Layer color encoded as a hex string.

Output:

- None.

_validate_common_paths(self)

Validates and create the project output folder used by export and compilation.

Inputs:

- None.

Output:

- Path for the output directory, or None if validation fails.

_write_manifest(self, manifest_path, vector_export_dir)

Writes the project manifest describing exported layers and viewer defaults.

Inputs:

- `manifest_path`: Output path for the manifest JSON file.
- `vector_export_dir`: Directory containing exported vector GeoJSON files.

Output:

- None.

`_export_selected_vectors(self, show_messages=True)`

Exports the selected vector layers to GeoJSON, attaches style metadata to it, and writes the manifest.

Inputs:

- `show_messages`: Whether to show user-facing success or warning dialogs.

Output:

- Path to the vector export directory, or None if export cannot proceed.

`convert_selected_vectors(self)`

Triggers a manual vector export from the dialog button.

Inputs:

- None.

Output:

- None.

Conversion and Compilation Methods

`run_las_to_tif_conversion(self)`

Validates the raster-conversion inputs and start the background conversion worker.

Inputs:

- None.

Output:

- None.

`compile_project(self)`

Validates inputs, export vectors, and starts the full Potree project compilation.

Inputs:

- None.

Output:

- None.

CompileProjectWorker

CompileProjectWorker runs full Potree project compilation in a background thread, including point cloud conversion, runtime library copying, vector export integration, and HTML generation.

Signals

finished

Emits a result payload when compilation succeeds.

Output:

- dict: currently containing the compiled output_dir.

failed

Emits a readable error message when compilation fails.

Output:

- str: containing the failure message.

log_message

Streams log lines back to the dialog while compilation is running.

Output:

- str: containing one log message.

Methods

__init__(self, potreeconverter_path, input_path, output_dir, project_name, vector_data_dir, pointcloud_display_mode, point_radius, projection, cesium_map, cesium_map_sea_level)

Stores all project compilation options for threaded execution.

Inputs:

- potreeconverter_path: Path to the PotreeConverter executable.
- input_path: Path to the source LAS/LAZ file.
- output_dir: Project output directory.

- `project_name`: Requested Potree project name.
- `vector_data_dir`: Directory containing exported vector GeoJSON files.
- `pointcloud_display_mode`: Default point cloud display mode in the generated viewer.
- `point_radius`: Radius used for point-vector overlays.
- `projection`: Optional projection string, typically an EPSG auth id.
- `cesium_map`: Whether Cesium map support should be enabled.
- `cesium_map_sea_level`: Elevation offset applied to the Cesium base map.

Output:

- None.

run(self)

Call the core compilation routine and emit success, failure, and log messages.

Inputs:

- None directly. Uses worker state set in `__init__`.

Output:

- None directly. (On success, emits `finished({"output_dir": ...})`, On failure, emits `failed(...)` signals.)

A.I.III potreecraft_geojson_reader.py

simple_geojson_reader

`simple_geojson_reader` reads a GeoJSON file, stores lightweight metadata, and extracts geometry data into the global Potree overlay feature collections used during HTML generation.

Methods

__init__(self, filepath)

Loads a GeoJSON file and cache basic metadata about it, which includes populating `name`, `crs`, `geomtype`, `feature_count`, `fields`. Initially creates an empty `feature_list`.

Inputs:

- `filepath`: Path to the GeoJSON file.

Output:

- None.

print_metadata(self)

Writes a short metadata summary to the log.

Inputs:

- None.

Output:

- None.

extract_coordinates(self)

This performs a walk through the GeoJSON features and gather supported geometry types into Potree-friendly coordinate structures – collects line features to `lns_gjs_feature_list`, point features to `pts_gjs_feature_list` and polygon features to `ply_gjs_feature_list`.

Inputs:

- None directly. (Uses the GeoJSON file referenced by `self.filepath`.)

Output:

- None directly. (Edits HTML structure.)

potree_html_generator

`potree_html_generator` writes the final ``potree_main.html`` file used to visualize the converted point cloud and vector overlays.

Methods

write_potree_html(cls, pointcloud_name, pointcloud_display_mode="rgb", point_radius=5.0, fallback_projection="", cesium_map=False, cesium_map_sea_level=0.0, output_dir=None)

Generate the Potree HTML entry file using either the default template or the Cesium-enabled template. The function is responsible for writing the `potree_main.html` into `output_dir`. It also appends the generated vector class and data payload to the HTML, and falls back to the non-Cesium template if the Cesium runtime is not present.

Inputs:

- `pointcloud_name`: Name of the point cloud project to load in the viewer.
- `pointcloud_display_mode`: Default point-cloud attribute mode, such as `rgb`, `intensity`, or `elevation`.
- `point_radius`: Radius used for rendered point-vector overlays.
- `fallback_projection`: Projection string used by the Cesium template when needed.
- `cesium_map`: Whether Cesium map support should be enabled.
- `cesium_map_sea_level`: Elevation offset applied in the Cesium template.
- `output_dir`: Optional output directory where `potree_main.html` will be written.

Output:

- None (directly).

A.I.IV potreecraft_lasreader.py

FlatLas Function Descriptions

The FlatLas class reads a LAS/LAZ point cloud, converts the selected value channel into a regular raster grid, and writes that grid to a GeoTIFF file.

Methods

__init__(self, input_path, output_dir, output_epsg, raster_mode)

This stores the source file, output location, target CRS, and rasterization mode.

Inputs:

- `input_path`: Path to the input LAS or LAZ file.
- `output_dir`: Directory where the GeoTIFF should be written.
- `output_epsg`: EPSG code to embed in the output GeoTIFF.
- `raster_mode`: Requested raster mode, for example `intensity`, `rbg` or `elevation`.

Output:

- None.

read_las(self)

The function which loads the point-cloud coordinates and supported attributes into memory. It has native .las support, but can only process .laz when a compatible laspy backend is available. It's job is to populate self.x, self.y, self.z, and self.intensity values.

Inputs:

- None directly. (Uses self.input_path).

Output:

- None.

interpolate_las(self)

This is an aggregate function to calculate point values onto a fixed-size raster grid and compute its affine transform.

Inputs:

- None directly. (Uses loaded point-cloud arrays from read_las()).

Output:

- Tuple (transform, zi), where *transform* is an affine transform created with rasterio.transform.from_origin, and *zi* is a numpy array containing the rasterized values.

output_path(self)

This builds the destination GeoTIFF filename from the current settings.

Inputs:

- None.

Output:

- Path to the target .tif file.

save_tif(self, transform, zi)

Writes the raster array to disk as a GeoTIFF.

Inputs:

- transform: Affine transform describing raster georeferencing.
- zi: Raster data array to save.

Output:

- Path to the written GeoTIFF file.

A.I.V potreecraft_core.py

Module Summary

potreecraft_core.py contains the non-UI project compilation helpers used by the QGIS plugin. It validates inputs, manages runtime library copying, synchronizes exported vector data, resolves the point-cloud name, and runs PotreeConverter plus HTML generation.

Functions

_emit(log_callback, message)

Sends a log message to the supplied callback when logging is enabled. (Note: Does absolutely nothing when *log_callback* is None.)

Inputs:

- *log_callback*: Optional callable that accepts a single string message.
- *message*: Log message text to emit.

Output:

- None.

prepare_vectors_folder(source_vector_dir, output_dir)

Copies GeoJSON vector exports into the final Potree project folder and removes stale files from earlier runs.

Inputs:

- *source_vector_dir*: Directory containing source .geojson files.
- *output_dir*: Root Potree project output directory.

Output:

- Tuple (*vectors_output_dir*, *copied_files*) where:
 - *vectors_output_dir*: Path to the destination *vectors* folder inside the project.
 - *copied_files*: Number of GeoJSON files copied or retained.

resolve_pointcloud_name(output_dir, requested_name)

Determines the point-cloud project name used by the generated HTML. Returns *requested_name* immediately if it is not empty. If no name is requested, it tries to infer

the name from the ``pointclouds`` subdirectory. In case this fails as well, it returns an empty string if the name cannot be resolved unambiguously.

Inputs:

- `output_dir`: Root Potree project output directory.
- `requested_name`: Explicit project name requested by the caller.

Output:

- `str`: containing the resolved point-cloud name.

copy_plugin_jslibs(output_dir, log_callback=None)

Copies PotreeCraft-specific runtime JavaScript libraries into the compiled projekt, such as Cesium version 1.83 and the r109 three.js build. Raises *FileNotFoundError* if required plugin library folders are missing.

Inputs:

- `output_dir`: Root Potree project output directory.
- `log_callback`: Optional logging callback for status messages.

Output:

- `None`.

compile_potree_project(*, potreeconverter_path, input_path, output_dir, project_name, vector_data_dir, pointcloud_display_mode="rgb", point_radius=5.0, projection="", cesium_map=False, cesium_map_sea_level=0.0, log_callback=None)

Runs the full Potree project build pipeline from point-cloud conversion through final HTML generation. This function launches PotreeConverter as a subprocess, and also verifies that the generated ``libs`` directory is populated. The process also copies plugin runtime JavaScript dependencies into the output alongside the vector GeoJSON files into the project and validates that at least one exists. Finally it resolves the final point-cloud name and calls ``generate_potree_html(...)``. The function raises exceptions when validation or build steps fail.

Inputs:

- `potreeconverter_path`: Path to the PotreeConverter executable.
- `input_path`: Path to the source LAS/LAZ point-cloud file.

- `output_dir`: Root output directory for the compiled Potree project.
- `project_name`: Requested point-cloud project name.
- `vector_data_dir`: Directory containing exported vector GeoJSON files.
- `pointcloud_display_mode`: Default point-cloud display mode in the generated viewer.
- `point_radius`: Radius used for rendered point-vector overlays.
- `projection`: Optional projection string passed to PotreeConverter and HTML generation.
- `cesium_map`: Whether Cesium map support should be enabled in the generated HTML.
- `cesium_map_sea_level`: Elevation offset applied to the Cesium map.
- `log_callback`: Optional callback for streaming build log messages.

Output:

- `int`, returning 0 on success.

A.II JSON and CSV data

A.II.I Hatri-dl-3-1-2026-cameras.json

```
{
  "dji_fc3682_4000_2250_brown_0.6666": {
    "projection_type": "brown",
    "width": 4000,
    "height": 2250,
    "focal_x": 0.6493349021020803,
    "focal_y": 0.6493349021020803,
    "c_x": -0.00364640912002432,
    "c_y": 0.0014308454855519182,
    "k1": 0.05134432106710347,
    "k2": -0.08661699579502441,
    "p1": -0.0005759111377238758,
    "p2": -0.0003061968194534967,
    "k3": 0.03316994433795165
  }
}
```

A.II.II Hatri-dl-3-1-2026-camera-params.json

```
{
```

```

"dji_fc3682_4000_2250_brown_0.6666": {
  "projection_type": "brown",
  "width": 4000,
  "height": 2250,
  "focal_x": 0.6493349021020803,
  "focal_y": 0.6493349021020803,
  "c_x": -0.00364640912002432,
  "c_y": 0.0014308454855519182,
  "k1": 0.05134432106710347,
  "k2": -0.08661699579502441,
  "p1": -0.0005759111377238758,
  "p2": -0.0003061968194534967,
  "k3": 0.03316994433795165
}
}

```

A.II.III Hatri-dl-3-1-2026-camera-shots.geojson

```

{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "properties": {
        "filename": "dji_fly_20260301_145018_99_1772373304919_timed.jpg",
        "camera": "v2 dji_fc3682_4000_2250_brown_0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376619,
        "translation": [
          353350.41602617514,
          5280083.228430791,
          80.79828843035642
        ],
        "rotation": [
          -2.062386147730597,
          -2.3414873778502048,
          -0.0539900359774499
        ]
      },
      "geometry": {
        "type": "Point",
        "coordinates": [
          19.046926185625843,
          47.65751384893861,

```

```

            80.79828843035642
        ]
    }
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145048_109_1772373304914_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376648,
        "translation": [
            353365.7446485134,
            5280062.516002245,
            81.46692492105984
        ],
        "rotation": [
            -2.7444549697821943,
            1.4728570545129411,
            0.010779388755545878
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.04713715711671,
            47.65733106669583,
            81.46692492105984
        ]
    }
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145052_110_1772373304914_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376651,
        "translation": [
            353392.18150357256,
            5280077.6381371245,

```

```

        81.98507985134408
    ],
    "rotation": [
        -2.6034575003557703,
        1.741232647798743,
        -0.006625213590998885
    ]
},
"geometry": {
    "type": "Point",
    "coordinates": [
        19.047483958340504,
        47.65747304329282,
        81.98507985134408
    ]
}
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145106_115_1772373304911_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376666,
        "translation": [
            353405.3698217814,
            5280030.208971166,
            82.66321006502417
        ],
        "rotation": [
            -1.6576291218391397,
            -2.5948532242609637,
            -0.0424958791591536
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.047675399808337,
            47.65704952488811,
            82.66321006502417
        ]
    }
}

```

```

},
{
  "type": "Feature",
  "properties": {
    "filename": "dji_fly_20260301_145036_105_1772373304916_timed.jpg",
    "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
    "focal": 0.6493349021020803,
    "width": 4000,
    "height": 2250,
    "capture_time": 1772376636,
    "translation": [
      353264.6273292863,
      5280028.306560576,
      83.13941436197511
    ],
    "rotation": [
      -2.1948011655063717,
      2.1774099299423986,
      0.060124775442878774
    ]
  },
  "geometry": {
    "type": "Point",
    "coordinates": [
      19.045802775950495,
      47.65700051455378,
      83.13941436197511
    ]
  }
},
{
  "type": "Feature",
  "properties": {
    "filename": "dji_fly_20260301_145100_113_1772373304912_timed.jpg",
    "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
    "focal": 0.6493349021020803,
    "width": 4000,
    "height": 2250,
    "capture_time": 1772376660,
    "translation": [
      353446.45470427355,
      5280062.875456534,
      84.42419515371715
    ],
    "rotation": [

```

```

        -1.504675471690579,
        -2.6944222639849134,
        -0.07330941371737473
    ]
},
"geometry": {
    "type": "Point",
    "coordinates": [
        19.048211282786344,
        47.65735258544225,
        84.42419515371715
    ]
}
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145058_112_1772373304913_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376657,
        "translation": [
            353444.7456185162,
            5280082.819293558,
            84.43283410379011
        ],
        "rotation": [
            -0.5126134376331649,
            -3.0532585385037505,
            -0.04829962041616372
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.048181848546836,
            47.657531543262536,
            84.43283410379011
        ]
    }
}
},
{
    "type": "Feature",

```

```

    "properties": {
      "filename": "dji_fly_20260301_145104_114_1772373304912_timed.jpg",
      "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
      "focal": 0.6493349021020803,
      "width": 4000,
      "height": 2250,
      "capture_time": 1772376663,
      "translation": [
        353429.49295424344,
        5280044.927255185,
        83.57531105433529
      ],
      "rotation": [
        -1.6563113587660396,
        -2.595974473724766,
        -0.06530204863429638
      ]
    },
    "geometry": {
      "type": "Point",
      "coordinates": [
        19.047991540676076,
        47.65718734410374,
        83.57531105433529
      ]
    }
  },
  {
    "type": "Feature",
    "properties": {
      "filename": "dji_fly_20260301_144958_92_1772373304922_timed.jpg",
      "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
      "focal": 0.6493349021020803,
      "width": 4000,
      "height": 2250,
      "capture_time": 1772376598,
      "translation": [
        353333.8667467095,
        5280128.329996568,
        81.22403250913715
      ],
      "rotation": [
        -2.567410179255783,
        1.80271716167335,
        0.0766326440573706
      ]
    }
  }
}

```

```

    ]
  },
  "geometry": {
    "type": "Point",
    "coordinates": [
      19.04669078260054,
      47.65791567315746,
      81.22403250913715
    ]
  }
},
{
  "type": "Feature",
  "properties": {
    "filename": "dji_fly_20260301_145016_98_1772373304920_timed.jpg",
    "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
    "focal": 0.6493349021020803,
    "width": 4000,
    "height": 2250,
    "capture_time": 1772376615,
    "translation": [
      353379.58753218985,
      5280088.379280038,
      81.26392162293067
    ],
    "rotation": [
      -2.058441573412298,
      -2.340891689026593,
      -0.07781895799692347
    ]
  },
  "geometry": {
    "type": "Point",
    "coordinates": [
      19.04731273021218,
      47.657566779158664,
      81.26392162293067
    ]
  }
},
{
  "type": "Feature",
  "properties": {
    "filename": "dji_fly_20260301_145034_104_1772373304917_timed.jpg",
    "camera": "v2 dji fc3682 4000 2250 brown 0.6666",

```

```

        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376633,
        "translation": [
            353247.3334378997,
            5280047.2191482,
            83.53982234421782
        ],
        "rotation": [
            -0.971916312810189,
            2.9345686494172183,
            0.033800213116848506
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.045566247958774,
            47.657166663516,
            83.53982234421782
        ]
    }
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145012_97_1772373304920_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376612,
        "translation": [
            353403.99602414446,
            5280097.162599747,
            82.05835924769542
        ],
        "rotation": [
            -2.0609362730468352,
            -2.342959890413912,
            -0.09592815810647505
        ]
    },
    "geometry": {

```

```

        "type": "Point",
        "coordinates": [
            19.047634661662105,
            47.65765129390552,
            82.05835924769542
        ]
    },
    {
        "type": "Feature",
        "properties": {
            "filename": "dji_fly_20260301_145028_102_1772373304918_timed.jpg",
            "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
            "focal": 0.6493349021020803,
            "width": 4000,
            "height": 2250,
            "capture_time": 1772376627,
            "translation": [
                353259.4745194971,
                5280072.724502762,
                82.94085935600205
            ],
            "rotation": [
                1.6902786214129641,
                2.632556924229431,
                -0.011417995928626799
            ]
        },
        "geometry": {
            "type": "Point",
            "coordinates": [
                19.045719282638427,
                47.657398773703754,
                82.94085935600205
            ]
        }
    },
    {
        "type": "Feature",
        "properties": {
            "filename": "dji_fly_20260301_145040_106_1772373304916_timed.jpg",
            "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
            "focal": 0.6493349021020803,
            "width": 4000,
            "height": 2250,

```

```

        "capture_time": 1772376639,
        "translation": [
            353289.54342857446,
            5280022.601590327,
            82.61243894547366
        ],
        "rotation": [
            -2.7764023702134137,
            1.358639810596562,
            0.0569508829484705
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.046136319954208,
            47.65695486277384,
            82.61243894547366
        ]
    }
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145112_117_1772373304910_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376672,
        "translation": [
            353350.9701837803,
            5280004.706169711,
            82.47377693902982
        ],
        "rotation": [
            -2.079105634370652,
            -2.2786319205943566,
            0.004642743353437419
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.04695990335745,

```

```

        47.65680786364746,
        82.47377693902982
    ]
}
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145024_101_1772373304918_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376624,
        "translation": [
            353290.3162508739,
            5280076.023905813,
            81.6175678973112
        ],
        "rotation": [
            -2.080868413555337,
            -2.351321783022693,
            0.00002612759605706283
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.046128677033643,
            47.657435437578236,
            81.6175678973112
        ]
    }
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145110_116_1772373304911_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376669,
        "translation": [
            353378.750568112,

```

```

        5280016.879508991,
        82.29703859500799
    ],
    "rotation": [
        -1.6599637739880673,
        -2.5969304892718745,
        -0.014809931593029513
    ]
},
"geometry": {
    "type": "Point",
    "coordinates": [
        19.047325571856188,
        47.65692362816103,
        82.29703859500799
    ]
}
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145046_108_1772373304915_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376645,
        "translation": [
            353339.03819169814,
            5280048.763456997,
            81.44008509197478
        ],
        "rotation": [
            -2.576714221938959,
            1.7217222467201894,
            0.020020788751449005
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.046786309928063,
            47.657201343986664,
            81.44008509197478
        ]
    ]
}

```

```

    }
  },
  {
    "type": "Feature",
    "properties": {
      "filename": "dji_fly_20260301_145116_118_1772373304909_timed.jpg",
      "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
      "focal": 0.6493349021020803,
      "width": 4000,
      "height": 2250,
      "capture_time": 1772376675,
      "translation": [
        353321.96158015734,
        5280001.777051921,
        82.70913960924389
      ],
      "rotation": [
        -2.9978606580833755,
        -0.6871160439885843,
        0.0011095878672439904
      ]
    },
    "geometry": {
      "type": "Point",
      "coordinates": [
        19.046574787484083,
        47.65677494814678,
        82.70913960924389
      ]
    }
  },
  {
    "type": "Feature",
    "properties": {
      "filename": "dji_fly_20260301_145010_96_1772373304920_timed.jpg",
      "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
      "focal": 0.6493349021020803,
      "width": 4000,
      "height": 2250,
      "capture_time": 1772376610,
      "translation": [
        353417.00141589536,
        5280115.909157461,
        83.13115825180367
      ],

```

```

        "rotation": [
            -1.3337884169479772,
            -2.821693540240183,
            -0.10752778794720697
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.047801477429356,
            47.65782281882164,
            83.13115825180367
        ]
    }
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145054_111_1772373304913_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376654,
        "translation": [
            353420.161310864,
            5280089.2652064,
            83.20633449721818
        ],
        "rotation": [
            1.5931325060010826,
            -2.6844872661188615,
            -0.0070228740055962675
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.047852470147426,
            47.65758393905232,
            83.20633449721818
        ]
    }
},
{

```

```

"type": "Feature",
"properties": {
  "filename": "dji_fly_20260301_145004_94_1772373304921_timed.jpg",
  "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
  "focal": 0.6493349021020803,
  "width": 4000,
  "height": 2250,
  "capture_time": 1772376603,
  "translation": [
    353383.5294134677,
    5280143.688262778,
    82.75495916650657
  ],
  "rotation": [
    1.9125411645018386,
    -2.4635174295705267,
    -0.07309786764160407
  ]
},
"geometry": {
  "type": "Point",
  "coordinates": [
    19.04734664556765,
    47.658065038133195,
    82.75495916650657
  ]
}
},
{
  "type": "Feature",
  "properties": {
    "filename": "dji_fly_20260301_145042_107_1772373304915_timed.jpg",
    "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
    "focal": 0.6493349021020803,
    "width": 4000,
    "height": 2250,
    "capture_time": 1772376642,
    "translation": [
      353314.194035028,
      5280032.8964372855,
      81.83092764700704
    ],
    "rotation": [
      -2.7779473061695064,
      1.3592496495230846,

```

```

        0.03845077593145294
    ]
},
"geometry": {
    "type": "Point",
    "coordinates": [
        19.046460960981822,
        47.65705302803569,
        81.83092764700704
    ]
}
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145022_100_1772373304919_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376621,
        "translation": [
            353320.49506497703,
            5280079.358146663,
            80.86359837215763
        ],
        "rotation": [
            -2.070883747161527,
            -2.345133752279319,
            -0.027084608847459925
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.046529236950438,
            47.657472262993466,
            80.86359837215763
        ]
    }
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_144954_91_1772373304923_timed.jpg",

```

```

        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376594,
        "translation": [
            353325.1534452156,
            5280125.23286739,
            81.27521798899359
        ],
        "rotation": [
            -2.561061257061799,
            1.7976107417534877,
            0.07203440778610637
        ]
    },
    "geometry": {
        "type": "Point",
        "coordinates": [
            19.046575847080916,
            47.65788584702438,
            81.27521798899359
        ]
    }
},
{
    "type": "Feature",
    "properties": {
        "filename": "dji_fly_20260301_145030_103_1772373304917_timed.jpg",
        "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
        "focal": 0.6493349021020803,
        "width": 4000,
        "height": 2250,
        "capture_time": 1772376630,
        "translation": [
            353238.10363759473,
            5280060.8288739845,
            84.08779443686284
        ],
        "rotation": [
            -1.8505383554648098,
            2.4950533015511707,
            0.05144989638455595
        ]
    },

```

```

    "geometry": {
      "type": "Point",
      "coordinates": [
        19.04543883122777,
        47.657286955271644,
        84.08779443686284
      ]
    }
  },
  {
    "type": "Feature",
    "properties": {
      "filename": "dji_fly_20260301_145000_93_1772373304922_timed.jpg",
      "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
      "focal": 0.6493349021020803,
      "width": 4000,
      "height": 2250,
      "capture_time": 1772376600,
      "translation": [
        353356.39816627215,
        5280135.677518535,
        81.63751220075281
      ],
      "rotation": [
        2.5597668129709574,
        -1.8002149893025592,
        -0.06876067171543311
      ]
    },
    "geometry": {
      "type": "Point",
      "coordinates": [
        19.046988212762308,
        47.65798685282525,
        81.63751220075281
      ]
    }
  },
  {
    "type": "Feature",
    "properties": {
      "filename": "dji_fly_20260301_145006_95_1772373304921_timed.jpg",
      "camera": "v2 dji fc3682 4000 2250 brown 0.6666",
      "focal": 0.6493349021020803,
      "width": 4000,

```

```

    "height": 2250,
    "capture_time": 1772376607,
    "translation": [
        353408.9539893078,
        5280137.631625278,
        83.46887391543096
    ],
    "rotation": [
        0.09998314279350108,
        -3.1182963512228747,
        -0.10188642299478404
    ]
},
"geometry": {
    "type": "Point",
    "coordinates": [
        19.047687080990485,
        47.65801633478694,
        83.46887391543096
    ]
}
}
]
}

```

A.II.IV overlap_report.csv

```

src_fid,src_id,area_m2,l_fwd,l_back,l_front,q_left,q_right,q_side,dist_fwd,dist_back,dist_left,dist_right,ang_fwd,ang_back,ang_left,ang_right,risk_class
0,dji_fly_20260301_145018_99_1772373304919_timed.jpg,8100.0000000000931,55.17352852873146,54.57096531481809,55.17352852873146,52.59000217334487,68.27878098704147,68.27878098704147,29.626030381641325,30.173559621707533,48.04725865473929,25.770484087073218,0.0,2.6432710924441483,10.136216034861208,26.49041290279815,terrible
2,dji_fly_20260301_145048_109_1772373304914_timed.jpg,8099.9999999994412,58.821146591158666,54.30796459635994,58.821146591158666,68.27878098714243,59.25490385023251,68.27878098714243,29.338107076721116,30.042741778997144,25.770484087073218,47.45882766251782,32.07286459592577,2.523674872979163,6.734019146640201,13.862983842917059,terrible
4,dji_fly_20260301_145052_110_1772373304914_timed.jpg,8099.9999999990687,54.201136992450024,53.09707421265944,54.201136992450024,76.42857512812881,56.06400263726397,76.42857512812881,30.302825653270364,30.459662591599297,16.554176985590086,49.23406151739667,0.0,7.204537374698531,26.97437840923441,7.026133076522838,terrible
6,dji_fly_20260301_145106_115_1772373304911_timed.jpg,8100.0000000002794,56.313965943358085,54.24850251934241,56.313965943358085,56.06400263718017,0.0,56.06400263718017,28.261808970826582,29.773391617815463,49.23406151739667,-1,0.0,4.789488409296617,15.849373342716072,-1,terrible

```

8,dji_fly_20260301_145036_105_1772373304916_timed.jpg,8100.000000007451,61.3984078579465
 94,61.44906927552171,61.44906927552171,55.89326978025899,0.0,55.89326978025899,25.563692
 14749514,25.630235978133353,44.720741068175734,-
 1,0.0,34.66328315374982,19.51371400323829,-1,terrible
 10,dji_fly_20260301_145100_113_1772373304912_timed.jpg,8099.999999988824,0.0,61.63483175
 4351566,61.634831754351566,52.74884826622501,0.0,52.74884826622501,-
 1,24.697646942993206,37.25678973172055,-1,-1,0.0,1.7233660640317472,-1,terrible
 12,dji_fly_20260301_145058_112_1772373304913_timed.jpg,8100.0,0.0,61.46691989668725,61.4
 6691989668725,0.0,66.32056653819242,66.32056653819242,-1,25.418120515748623,-
 1,20.01914756990482,-1,0.0,-1,19.5900079997899,terrible
 14,dji_fly_20260301_145104_114_1772373304912_timed.jpg,8100.000000010245,61.634831754136
 83,56.313965943329286,61.63483175413683,54.66577905137077,0.0,54.66577905137077,24.69764
 6942993206,28.261808970826582,45.31431787315492,-
 1,0.0,15.229895637676494,34.73321042233438,-1,terrible
 16,dji_fly_20260301_144958_92_1772373304922_timed.jpg,8099.99999998603,63.82987383646217
 ,85.67697866250487,85.67697866250487,0.0,52.59000217344162,52.59000217344162,23.70179096
 7470757,9.248388659759227,-1,48.04725865473929,0.0,1.506468586325978,-
 1,2.0885703404343334,weak
 18,dji_fly_20260301_145016_98_1772373304920_timed.jpg,8100.000000007451,59.9998019275761
 44,55.17352852862956,59.999801927576144,44.78386149091413,76.42857512792465,76.428575127
 92465,25.94358964376169,29.626030381641325,52.68285100250365,16.554176985590086,0.0,9.77
 753592953811,6.326671044157937,29.74869886115569,terrible
 20,dji_fly_20260301_145034_104_1772373304917_timed.jpg,8100.0,59.74785297051256,0.0,59.7
 4785297051256,74.02469324375744,61.44906927557823,74.02469324375744,28.250741361318386,-
 1,16.44607155212136,25.630235978133353,30.716720844552658,-
 1,0.3163537926829709,8.612300658944632,terrible
 22,dji_fly_20260301_145012_97_1772373304920_timed.jpg,8099.999999997206,0.0,59.999801927
 663526,59.999801927663526,50.85283333970907,38.46217709114193,50.85283333970907,-
 1,25.94358964376169,40.776108347471414,54.58027400532139,-
 1,0.0,26.77575389217544,31.286573055254706,terrible
 24,dji_fly_20260301_145028_102_1772373304918_timed.jpg,8099.999999988824,53.023035478241
 96,56.94461465473842,56.94461465473842,0.0,59.747852970595,59.747852970595,31.0211238725
 7428,24.461236060188643,-1,28.250741361318386,0.0,22.995337918391602,-
 1,31.56162005682628,terrible
 26,dji_fly_20260301_145040_106_1772373304916_timed.jpg,8099.999999988824,59.508894753437
 04,0.0,59.50889475343704,52.55418213898737,42.635274541703595,52.55418213898737,26.71691
 0929293405,-1,53.43378392493545,38.5347202409412,0.0,-
 1,23.495700897840727,34.617503352867224,terrible
 28,dji_fly_20260301_145112_117_1772373304910_timed.jpg,8099.999999994412,53.378498695310
 19,56.30091467881895,56.30091467881895,57.04149054691543,0.0,57.04149054691543,30.333855
 37466635,29.159321884980045,45.649492977635454,-
 1,0.0,17.897141419350096,8.509146845911713,-1,terrible
 30,dji_fly_20260301_145024_101_1772373304918_timed.jpg,8099.999999998137,54.397482927975
 88,53.023035478158,54.39748292797588,0.0,46.65293625720274,46.65293625720274,30.36578697

7406703,31.02112387257428,-1,49.30174609686481,0.0,0.19843387043443386,-
1,22.666736386856023,terrible
32,dji_fly_20260301_145110_116_1772373304911_timed.jpg,8100.000000005588,54.248502519329
45,53.37849869516181,54.24850251932945,59.254903850150754,0.0,59.254903850150754,29.7733
91617815463,30.33385537466635,47.45882766251782,-
1,0.0,2.936207443578477,10.692198325115163,-1,terrible
34,dji_fly_20260301_145046_108_1772373304915_timed.jpg,8100.000000006519,54.307964596278
765,54.42745300557361,54.42745300557361,60.6955055823912,57.041490546864665,60.695505582
3912,30.042741778997144,29.481958475088078,35.77938890891663,45.649492977635454,0.0,5.31
8528167600804,3.9733284557712647,12.092464934312629,terrible
36,dji_fly_20260301_145116_118_1772373304909_timed.jpg,8099.999999996275,56.300914678760
01,10.398675549458206,56.30091467876001,68.55186688599605,0.0,68.55186688599605,29.15932
1884980045,63.181556702187564,32.07767796753808,-
1,0.0,30.596604871306116,8.249082061223739,-1,terrible
37,dji_fly_20260301_145010_96_1772373304920_timed.jpg,8100.000000007451,0.0,64.810621834
55914,64.81062183455914,56.88312582583626,61.339816087087605,61.339816087087605,-
1,22.818605190850366,23.167765904804277,26.83364074633178,-
1,18.90271636162689,16.018434802877763,29.58285342214188,terrible
39,dji_fly_20260301_145054_111_1772373304913_timed.jpg,8100.000000003725,61.466919896658
986,71.62319500138564,71.62319500138564,61.339816087115814,54.66577905142626,61.33981608
7115814,25.418120515748623,17.99324518435082,26.83364074633178,45.31431787315492,0.0,11.
345348265165974,21.45554503246867,26.577380571965573,weak
41,dji_fly_20260301_145004_94_1772373304921_timed.jpg,8099.999999999069,56.5693844587687
66,56.98265379091897,56.98265379091897,0.0,45.06624455630929,45.06624455630929,26.138916
056564643,28.292284618631417,-1,55.45539900431632,0.0,29.84896267150268,-
1,9.322660369573855,terrible
43,dji_fly_20260301_145042_107_1772373304915_timed.jpg,8100.000000002794,54.427453005564
146,59.50889475333441,59.50889475333441,46.652936257175924,68.55186688592937,68.55186688
592937,29.481958475088078,26.716910929293405,49.30174609686481,32.07767796753808,0.0,9.8
97919952538407,3.59345000441582,18.549905614127965,terrible
45,dji_fly_20260301_145022_100_1772373304919_timed.jpg,8100.0,54.57096531480137,54.39748
292792313,54.57096531480137,52.37973834597218,60.695505582394055,60.695505582394055,30.1
73559621707533,30.365786977406703,46.11571596093739,35.77938890891663,0.0,1.065668248048
0923,13.16855952537837,23.84931843139405,terrible
47,dji_fly_20260301_144954_91_1772373304923_timed.jpg,8100.000000005588,85.6769786622520
1,0.0,85.67697866225201,0.0,52.379738345947544,52.379738345947544,9.248388659759227,84.0
9760259200726,-1,46.11571596093739,0.0,19.073626775476644,-1,25.365944898575204,weak
48,dji_fly_20260301_145030_103_1772373304917_timed.jpg,8099.999999996275,56.944614654628
56,0.0,56.94461465462856,0.0,74.02469324383748,74.02469324383748,24.461236060188643,-1,-
1,16.44607155212136,0.0,-1,-1,5.042674972910364,terrible
50,dji_fly_20260301_145000_93_1772373304922_timed.jpg,8099.999999980442,56.9826537910557
5,63.82987383644872,63.82987383644872,0.0,44.783861491063455,44.783861491063455,28.29228
4618631417,23.701790967470757,-1,52.68285100250365,0.0,1.611499196798947,-
1,9.668060476068465,terrible

52,dji_fly_20260301_145006_95_1772373304921_timed.jpg,8100.0,0.0,56.569384458739265,56.5
69384458739265,0.0,56.88312582587708,56.88312582587708,-1,26.138916056564643,-
1,23.167765904804277,-1,0.0,-1,33.72718111663724,terrible